Auswertung von Netzwerkfuzzern am Beispiel des MQTT Protokolls

Analyse der Performance von Netzwerkfuzzern unter Verwendung von Pulsar, AFLNet und boofuzz am Beispiel des MQTT Protokolls

B a c h e l o r a r b e i t

Hochschule für Angewandte Wissenschaften Hof
Fakultät Informatik
Studiengang Mobile Computing

Vorgelegt bei
Prof. Dr. Florian Adamsky
Alfons-Goppel-Platz 1
95028 Hof

Vorgelegt von
Sebastian Peschke
Mtr. Nr.: 00706319
Jahnstraße 6a
95030 Hof

Hof, 1. September 2024

Inhaltsverzeichnis

1 Einleitung **1**

2 Hintergrund **2**
2.1 Software-Testing Methoden 2
2.1.1 White-Box testing 2
2.1.2 Black-Box testing 3
2.1.3 Grey-Box testing 3
2.2 Fuzzing 4
2.2.1 Arten des Fuzzing 4
2.2.2 Generierung von Input 6
2.2.3 Intelligenz von Fuzzern 6
2.2.4 Strategien von Fuzzern 7
2.3 Grundlagen der TCP/IP-Kommunikation 8
2.4 Grundlagen des Mosquitto MQTT-Brokers 9

3 Related Work **15**

4 Analyse der Fuzzer Performance **17**
4.1 Fuzzing mit AFLNet 17
4.1.1 Setup 19
4.1.2 Analyse der Effektivität gesendeter Pakete 20
4.1.3 Analyse der Geschwindigkeit 22
4.1.4 Analyse der gefundenen Bugs 25
4.2 Fuzzing mit boofuzz 26
4.2.1 Setup 27
4.2.2 Analyse der Effektivität gesendeter Pakete 29
4.2.3 Analyse der gefundenen Bugs (Optional?) 34
4.3 Pulsar 34
4.3.1 Setup 34
4.3.2 Erkenntnisgewinn 35
4.4 Vergleich der erhobenen Metriken 35

5 Konklusion und Future Work **36**

A Listings **38**

Abbildungsverzeichnis

1 State Machine des Mosquitto MQTT-Brokers. Sie beschreibt alle Zustände, die der Broker ab Start der Applikation durchlaufen kann. Beginnend mit der Inintialisierung, über den zuhörenden Modus in dem der Broker auf eingehende Nachrichten wartet, bis hin zum Senden von Nachrichten an die Clients. Clients können sich mit dem Broker verbinden, Nachrichten abonnieren und Nachrichten senden. Der Broker kann Nachrichten an die Clients senden, die diese abonniert haben. 11

2 State Machine des Mosquitto MQTT-Protokolls. Sie beschreibt alle Zustände, die der Broker nach Empfangen einer Nachricht durchläuft. Beginnend mit der Überprüfung der Nachricht, über das Weiterleiten der Nachricht an die Abonnenten, bis hin zum Senden der Nachricht an die Clients. 12

3 Format einer MQTT (Message Queueing Telemetry Transport)-Nachricht am beispiel eines Connect-Pakets. 13

4 Architektur von AFL (American Fuzzy Lop)Net. Es wird die Architektur von AFLNet dargestellt. 18

5 Evaluation der Verbindungsaufbauten des ZuP (Zu untersuchendes Programm) Mosquitto. Die Log-Datei `mqtt-aflnet_stdout.log` wurde auf die Häufigkeit der Ausgaben von erfolgreichen und fehlgeschlagenen Verbindungsaufbauten analysiert. 20

6 Evaluation der Fehleranfälligkeit des ZuP Mosquitto. Die Log-Datei mqtt-aflnet_stdout.log wurde auf die Häufigkeit der Ausgaben von Fehlermeldungen analysiert. 21

7 Analyse der generierten Nachrichtensequenzen von AFLNet. Es wird die Anzahl der generierten Nachrichtensequenzen und die Anzahl der Nachrichtensequenzen, die an das ZuP gesendet wurden, analysiert. 22

8 Analyse der Geschwindigkeit von AFLNet. Es wird die Anzahl der generierten Testcases pro Sekunde und die Anzahl der Testcases, die an das ZuP gesendet wurden, analysiert. 23

9 Analyse der gefundenen Bugs von AFLNet. Es wird die Anzahl der gefundenen Bugs und die Art der Bugs analysiert. 24

10 Vergleich der gültigen Pakete bei zufälligem Fuzzing mit boofuzz ohne statisch definierte Felder 33

11 Durchschnittliche Ausführungsgeschwindigkeit von boofuzz ohne statisch definierte Felder 33

Abkürzungsverzeichnis

AFL	American Fuzzy Lop
DFA	Deterministic Finite Automaton
DNS	Domain Name System
FTP	File Transfer Protocol
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
M2M	Machine to Machine
MQTT	Message Queueing Telemetry Transport
pcap	packet capture
QoS	Quality of Service
SMTP	Simple Mail Transfer Protocol
TCP	Transmition Control Protocol
TLS	Transport Layer Security
ZuP	Zu untersuchendes Programm

Listings

1 Sanity-Checks des Brokers nach erhalten einer Publish Nachricht in der Funktion `handle__publish` aus `srchandle_publish.c` 38
2 Buffer Overflow in der Funktion `handle__connect` der Quellcodedatei src/handle_connect.c 38
3 Buffer Overflow in der Funktion `handle__publish` in der Quelldatei `src/handle_publish.c` des Mosquitto Brokers 39
4 Memory Leak in der Funktion `int connect__on_authorised` der Quellcodedatei `src/handle_subscribe.c` des Mosquitto Brokers 39

1 Einleitung

In der heutigen digital vernetzten Welt sind Netzwerke und ihre Protokolle das Rückgrat zahlreicher kritischer Anwendungen, von einfachen Webdiensten bis hin zu komplexen IoT (Internet of Things)-Systemen. Eine entscheidende Komponente der Netzwerksicherheit ist das Testen und die Absicherung dieser Protokolle gegen potenzielle Schwachstellen. Hierbei kommt die Technik des Fuzzing zum Einsatz, die durch die gezielte Überlastung eines Systems mit fehlerhaften oder unerwarteten Eingaben mögliche Sicherheitslücken aufdecken soll. Besonders relevant ist die Analyse von Netzwerkfuzzern, die speziell für die Sicherheitstests von Netzwerkprotokollen entwickelt wurden. Das MQTT-Protokoll, ein leichtgewichtiges Messaging-Protokoll für kleine Sensoren und mobile Geräte, stellt in vielen IoT- und M2M (Machine to Machine)-Anwendungen einen wichtigen Standard dar. Aufgrund seiner weitreichenden Nutzung und der oft sensiblen Natur der übertragenen Daten, ist es essenziell, die Sicherheit dieses Protokolls gründlich zu prüfen. Netzwerkfuzzer wie Pulsar, AFLNet und boofuzz bieten unterschiedliche Ansätze zur Durchführung solcher Tests. Pulsar fokussiert sich auf eine dynamische und effektive Fehlersuche durch einen Mix aus Netzwerk- und Protokoll-basiertem Fuzzing. AFLNet basiert auf dem bewährten Ansatz von AFL und passt diesen für Netzwerkprotokolle an, um eine effektive Testabdeckung zu gewährleisten. Boofuzz – eine Weiterentwicklung des etablierten Sulley-Fuzzers – bietet spezifische Funktionen für das Testen von Netzwerkdiensten und deren Protokollen. In dieser Arbeit wird die Performance dieser Netzwerkfuzzer im Kontext des MQTT-Protokolls detailliert analysiert. Ziel ist es, durch einen Vergleich der Fuzzing-Techniken und deren Effektivität Erkenntnisse zu gewinnen, die über die bloße Anwendung hinausgehen und zu einer fundierten Bewertung der getesteten Tools führen. Die Untersuchung fokussiert sich auf folgende Forschungsfragen:

Q1 Welchen Performanceunterschied gibt es unter den Netzwerkfuzzern?

Q2 Inwiefern sind gefundene Bugs reproduzierbar?

Durch die umfassende Betrachtung dieser Fragen soll ein fundierter Vergleich der Netzwerkfuzzer ermöglicht werden, der sowohl ihre Stärken als auch ihre Schwächen aufzeigt und wertvolle Erkenntnisse für die Weiterentwicklung und Anwendung solcher Tools liefert. Die Ergebnisse dieser Arbeit werden nicht nur zur Verbesserung der Sicherheitsüberprüfung des MQTT-Protokolls beitragen, sondern auch allgemeine Empfehlungen für den Einsatz von Fuzzern im Netzwerkbereich bieten.

2 Hintergrund

Folgendes Kapitel beschreibt die Grundlagen der in dieser Arbeit angewendeten Testing-Methoden und Herangehensweisen. Hierfür werden zunächst die Software-Testing-Methoden vorgestellt, die im Applikations- und System-Testing verwendet werden. Anschließend wird das Fuzzing als eine der wichtigsten Methoden des Software-Testings vorgestellt und kategorisiert. Zum weiteren Verständnis werden die Grundlagen des TCP (Transmition Control Protocol)-Protokolls erläutert. Darauf aufbauend werden die Grundlagen des ZuPs MQTT vorgestellt und die Funktionsweise des Protokolls erläutert, um eine ausreichende Grundlage zum Verständnis der Analyse zu schaffen.

2.1 Software-Testing Methoden

Das Fuzzing ist eine Software-Testing Methode zum Finden von Bugs und Sicherheitslücken in Applikationen. Es gibt verschiedene Arten von Software-Testing Methoden, die in der Software-Entwicklung eingesetzt werden. Die bekanntesten sind White-Box, Black-Box und Grey-Box Testing. Sie unterscheiden sich in der Art und Weise, wie sie die Software testen. Jede Methode hat ihre eigenen Vor- und Nachteile und wird in verschiedenen Situationen eingesetzt. In den folgenden Abschnitten werden die verschiedenen Software-Testing Methoden genauer erläutert.

2.1.1 White-Box testing

Es handelt sich um White-Box Testing, wenn der Tester Zugriff auf den Quellcode der Software hat. Diese Testingmethode wird auch als *Clear-Box*, *Transparent-Box* oder *Open-Box* Testing bezeichnet. Der Tester kann den Quellcode der Software analysieren und die Testfälle basierend auf dem Quellcode erstellen [1]. White-Box Testing wird normalerweise von Entwicklern durchgeführt, um sicherzustellen, dass der Code korrekt funktioniert und keine Fehler enthält. Es wird auch verwendet, um die Testabdeckung zu messen und sicherzustellen, dass alle Teile des Codes getestet wurden. White-Box Testing ist eine sehr effektive Methode, um Fehler in der Software zu finden, aber es erfordert auch eine gründliche Kenntnis des Quellcodes und der Software-Architektur. Zu White-Box Testing gehören mitunter das *Unit Testing*, bei dem einzelne Komponenten der Software getestet werden und *Path Testing* bei dem alle möglichen Programmpfade innerhalb eines zu testenden Abschnittes abgelaufen und überprüft werden sollen.
Der Vorteil dieser Testmethode besteht darin, dass hierbei komplexe und auch strukturelle

Fehler aufgedeckt werden können. Die Anwendung einer solchen Testmethode findet im optimalen Fall während der Entwicklung einer Anwendung statt.

2.1.2 Black-Box testing

Bei Black-Box Testing wird kein Zugriff auf den Quellcode und die interne Struktur der Application gebraucht. Das Ziel dieser Testmethode ist es zunächst mögliche Eingaben von Benutzern einer Software nachzuahmen [2]. Somit werden funktionalitäten einer Software getestet, ohne dass der Tester den Quellcode kennt. In der Regel werden hierbei nur bereits kompilierte Software-Module oder von außen Zugreifbare Endpunkte getestet. Anwendungsfälle einer solchen Herangehensweise entsprechen einem Penetrationstest oder eines Usability-Tests. Der Vorteil von Black-Box Testing besteht darin, dass das ZuP aus der Sicht des Answers getestet wird. Dabei wird die Software auf ihre Funktionalität und Benutzerfreundlichkeit getestet.
Black-Box Testing ist eine effektive Methode, um sicherzustellen, dass die Software den gestellten Anforderungen entspricht und keine offensichtlichen Fehler enthält. Es ist jedoch schwierig, komplexe Fehler zu finden, da der Tester keinen Zugriff auf den Quellcode hat und nicht weiß, wie die Software intern funktioniert.

2.1.3 Grey-Box testing

Grey-Box Testing kombiniert beide Aspekte der bereits genannten Ansätze. Hierbei werden Teile des ZuP offengelegt. Während beim White-Box Testing die interne Struktur oder der Code des Systems bekannt und getestet wird und beim Black-Box Testing nur die externen Funktionalitäten ohne Wissen über den internen Aufbau geprüft werden, ermöglicht Grey-Box Testing eine Balance zwischen diesen Ansätzen. Beim Grey-Box Testing hat der Tester teilweise Kenntnisse über die interne Struktur der Anwendung oder des Systems, verwendet diese Informationen jedoch hauptsächlich, um die Erstellung und Durchführung von Testszenarien zu optimieren, die auf der Benutzeroberfläche basieren [3]. Diese Methode erlaubt es, gezielte Testfälle zu entwerfen, die nicht nur auf den sichtbaren Eingaben und Ausgaben basieren, sondern auch auf dem Wissen über die zugrundeliegenden Datenflüsse, Algorithmen oder architektonischen Schwächen. Grey-Box Testing wird häufig in Szenarien eingesetzt, in denen eine vollständige Kenntnis der internen Struktur nicht notwendig oder verfügbar ist, jedoch ein gewisses Maß an Verständnis über die Architektur oder den Code notwendig ist, um tiefere Testfälle zu entwickeln. Diese Technik kann besonders nützlich sein bei der Validierung von Sicherheitsaspekten, der Integrationstests von komplexen Systemen oder der Optimierung der Testabdeckung bei großen Anwendungen. In der Praxis findet Grey-Box Testing Anwendung

in Bereichen wie Web-Applikationen, APIs und bei sicherheitskritischen Systemen, bei denen sowohl die Funktionalität als auch die interne Verarbeitung zuverlässig und sicher getestet werden müssen. Die Methode bietet somit eine ausgewogene Teststrategie, die die Vorteile von White-Box und Black-Box Testing vereint, um eine effektivere Fehlererkennung zu ermöglichen.

2.2 Fuzzing

Nochmal auf black box testing eingehen.
Fuzzing ist eine Black Box Testmethode, die darauf abzielt, Fehler in Software zu finden, indem zufällige oder semi-zufällige Daten als Eingabe verwendet werden [4]. Diese Daten werden in der Regel von einem Fuzzer generiert, der die Software mit den Daten füttert und auf unerwartetes Verhalten prüft. Fuzzing ist eine effektive Methode, um Fehler in Software zu finden, die durch unerwartete Eingaben verursacht werden. Es ist eine weit verbreitete Methode, um Sicherheitslücken in Software zu finden, die von Hackern ausgenutzt werden können. Fuzzing kann auch dazu verwendet werden, um die Stabilität und Zuverlässigkeit von Software zu testen und um sicherzustellen, dass sie korrekt funktioniert.
Nicht jeder Fuzzer ist gleich. Es gibt verschiedene Arten von Fuzzern, die unterschiedliche Strategien und Techniken verwenden, um Fehler in Software zu finden. Die Implementierung der Techniken und Strategien hängen von den Anforderungen und Zielen des Fuzzers ab. In den folgenden Abschnitten werden die verschiedenen Arten von Fuzzern, ihre Strategien und Techniken näher erläutert.

2.2.1 Arten des Fuzzing

Trotz dem, dass als Fuzzing eine Black-Box Testmethode gilt, gibt es verschiedene Arten von Fuzzing, die unterschiedliche Strategien und Techniken verwenden, um Fehler in Software zu finden. Die Arten des Fuzzing sind somit in drei Kategorien unterteilt [5]:

- Black-box Fuzzing
- White-box Fuzzing
- Grey-box Fuzzing

Jede Kategorie hat ihre eigenen Vor- und Nachteile und wird in den folgenden Abschnitten näher erläutert.
Im Black-box Fuzzing wird die Software ohne Kenntnis des ursprünglichen Quellcodes getestet. Der Fuzzer generiert zufällige oder semi-zufällige Eingaben und übergibt sie der Software.

Das Generieren der Eingaben erfolgt in der Regel durch Mutation von vorhandenen Eingaben oder durch Generierung neuer Eingaben und wird durch Regeln limitiert. Mit diesen Regeln wird sichergestellt, dass die Mehrheit der Eingaben nicht von der zu untersuchenden Software – aufgrund von bspw. Syntaxfehlern – abgelehnt wird. Bei der Durchführung des Fuzzing überwacht der Fuzzer das Verhalten der Software und prüft, ob unerwartetes Verhalten auftritt. Das Paradigma, das hierbei verfolgt wird, ist das Testen der Software ohne Kenntnis des Quellcodes, des Kontrollflusses und somit der internen Logik, um die Benutzung aus der Perspektive eines Nutzers der Software nachzustellen.
Die Vorteile dieser Herangehensweise sind, dass der Code nicht verfügbar sein muss und somit auch proprietäre Software getestet werden kann. Zudem werden Fehler gefunden, welche durch einen Nutzer der Software ausgelöst werden können. Der ausschlaggebende Nachteil dieser Vorgehensweise ist, dass die Eingaben aufgrund limitierter Informationen über die Software nicht zu komplexeren Bugs führen können. Das hat zufolge, dass die gefundenen Bugs meistens einfacher Natur sind und Eingaben oftmals nicht in tiefe Verzweigungen des Codes gelangen können [6].

Im Gegensatz zum Black-box Fuzzing wird im White-box Fuzzing der Quellcode der Software benötigt. Der Fuzzer generiert Eingaben, die auf der Analyse des Quellcodes basieren. Dabei werden die Eingaben so generiert, dass sie die verschiedenen Pfade und Verzweigungen des Codes abdecken. Das Ziel ist es, die Software mit Eingaben zu füttern, die potenzielle Fehler in den verschiedenen Teilen des Codes auslösen. Durch die Kenntnis des Quellcodes kann der Fuzzer gezieltere Eingaben generieren und somit auch komplexere Fehler finden. Die Vorteile dieser Methode sind, dass sie effektiver ist als Black-box Fuzzing, da sie gezieltere Eingaben generieren kann. Sie kann auch dazu verwendet werden, um spezifische Teile des Codes zu testen und um sicherzustellen, dass sie korrekt funktionieren. Der Nachteil dieser Methode ist, dass der Quellcode verfügbar sein muss, was bei proprietärer Software ein Problem darstellen kann. Zudem kann das White-box Fuzzing aufgrund der Komplexität des Codes und der Anzahl der möglichen Pfade und Verzweigungen sehr aufwendig sein [6].

Grey-box Fuzzing ist eine Kombination aus Black-box und White-box Fuzzing. Die Verwendung eines Gray-box Fuzzers erfordert keinen Zugriff auf Sourcecode, jedoch kann dieser optional verwendet werden. Unter Verwendung des Quellcodes kann die Performance des Fuzzers verbessert werden, indem gezieltere Eingaben generiert werden. Die Vorteile des Gray-box Fuzzing sind, dass es effektiver ist als Black-box Fuzzing, da es gezieltere Eingaben generieren kann. Außerdem ermöglicht dieser Ansatz auch das Analysieren von proprietärer Software aufgrund dessen, dass der Quellcode nicht für das Fuzzen von Software

notwendig ist [7]. Die Herangehensweise des Gray-box Fuzzing ist ähnlich zu dem, des Black-box Fuzzing. Jedoch können vor dem Kompilieren des Quellcodes Instruktionen des Fuzzers eingefügt werden, um das Fuzzing zu verbessern. Zudem wird unter bspw. AFL ein eigener Compiler verwendet, um Metadaten des Programms zu extrahieren und weitere Instrumentierungsanweisungen in den Code zu injizieren [8].

2.2.2 Generierung von Input

Die Generierung von Input ist ein wichtiger Bestandteil des Fuzzing-Prozesses. Hierbei unterscheiden sich die Fuzzer in ihrer Herangehensweise und den Techniken, die sie verwenden, um Eingaben zu generieren. Die Generierung von Input kann auf verschiedene Arten erfolgen, die in den folgenden Abschnitten näher erläutert werden.

Generationsbasierende [5] Fuzzer generieren Eingaben, indem sie eine Menge von Eingaben erstellen. Diese Eingaben basieren auf einem vom Anwender festgelegten Regelwerk. Diese Herangehensweise des Generierens von Eingaben ist effektiv, um die Software mit Eingaben zu versorgen, welche nicht bereits im Vorfeld von dem ZuP als fehlerhaft befunden werden sollen. Beispielsweise ist dies Vorteilhaft, wenn besonders Komplexe Synthax des Input vonnöten ist.

Mutationsbasierende [5] Fuzzer generieren Eingaben, indem sie vorhandene Eingaben, welche bereits im Vorfeld definiert wurden, verändern. Hierbei wird der Aufbau und die Struktur der Eingaben standardmäßig nicht berücksichtigt. Diese Veränderung wird Mutation genannt und kann auf verschiedene Arten erfolgen. Die Mutation kann beispielsweise durch das Hinzufügen, Entfernen oder Verändern von Bytes in der Eingabe erfolgen. Um die Güte der neu generierten Eingaben zu bestimmen, wird die Code Coverage – also die im Quellcode erreichte Tiefe – analysiert. Eingaben, die besonders viele Codesegmente erreichen werden als besonders gut gewertet und als bevorzugte Eingabe für weitere Mutationen verwendet.

2.2.3 Intelligenz von Fuzzern

Die Intelligenz verschiedener Fuzzer kann in zwei Kategorien [9] unterteilt werden:

- Dumme Fuzzer
- Intelligente Fuzzer

Zu den *dummen* Fuzzern gehören diejenigen, die das Feedback eines Programms nicht interpretieren. Hierzu werden die Eingaben zufällig generiert und an das ZuP übergeben. Aufgrund

dessen werden die Eingaben nicht auf ihre Gültigkeit oder Korrektheit überprüft. Dumme Fuzzer sind in der Regel einfach zu implementieren und erfordern keine speziellen Kenntnisse über das ZuP. Sie sind jedoch weniger effektiv als intelligente Fuzzer [9], da sie keine Informationen über das Verhalten des Programms sammeln und somit nicht in der Lage sind, gezieltere Eingaben zu generieren. Ein Vorteil der Verwendung von dummen Fuzzern ist, dass sie in der Regel schneller sind als intelligente Fuzzer. Das ist darauf zurückzuführen, dass aufgrund der nicht verwendeten Feedback-Schleife die Eingaben schneller generiert werden können.

Intelligente Fuzzer hingegen adaptieren die Generierung der Eingaben anhand des Feedbacks. Hierzu werden verschiedene Aspekte des Feedbacks analysiert, um gezieltere Eingaben zu generieren. Zu diesen Aspekten gehören beispielsweise die Reaktion des Programms auf die Eingabe, die Anzahl der ausgeführten Instruktionen oder die Anzahl der gefundenen Bugs. Der tatsächliche Fokus der Analyse hängt von der Implementierung des Fuzzers ab. Intelligente Fuzzer können das Verhalten des Programms überwachen und auf unerwartete Ereignisse reagieren [10]. Sie können auch Informationen über die Struktur des Programms sammeln und diese Informationen verwenden, um gezieltere Eingaben zu generieren.
Intelligente Fuzzer sind in der Regel effektiver als dumme Fuzzer, da sie gezieltere Eingaben generieren können. Sie sind jedoch auch komplexer zu implementieren und erfordern spezielle Kenntnisse über das ZuP.

2.2.4 Strategien von Fuzzern

Fuzzer können verschiedene Strategien verfolgen, um Eingaben zu generieren und das Verhalten der Software zu überwachen. Die Strategien können in zwei Kategorien unterteilt [5] werden:

- Abdeckung von Codepfaden
- Zielgerichtetes Fuzzen

Bei dem zielgerichteten Fuzzen werden nur bestimmte Teile des Codes getestet. Der Fuzzer generiert Eingaben, die auf bestimmten Kriterien basieren, um gezielt Fehler in diesen Teilen des Codes zu finden. Zu den Kriterien gehören beispielsweise die Anzahl der ausgeführten Instruktionen, die Anzahl der gefundenen Bugs oder die Reaktion des Programms auf die Eingabe. Bei dieser Technik handelt es sich in der Regel um einen White-Box Fuzzer. Um nur auf bestimmte Teile eines Programms zu fokussieren, wird der Quellcode benötigt [11].

Bei der Strategie der Codepfadabdeckung wird versucht, möglichst viele Pfade und Verzweigungen des Codes abzudecken. Der Fuzzer generiert Eingaben, die verschiedene Pfade und Verzweigungen des Codes abdecken, um potenzielle Fehler in verschiedenen Teilen des Codes zu finden. Diese Technik wird in der Regel von Black- und Grey-Box Fuzzern verwendet, da sie keine Kenntnis des Quellcodes benötigen und sich somit anderer Informationen bedienen müssen. Die Codepfadabdeckung ist eine effektive Methode, um sicherzustellen, dass große Teile des Codes getestet wurden [5]. Somit ist es ebenso möglich besonders komplexe Bugs und Schwachstellen in einem System zu finden, da die Eingaben besonders tief in den Programmcode gelangen.

2.3 Grundlagen der TCP/IP-Kommunikation

Die TCP/IP (Internet Protocol)-Kommunikation ist ein Protokoll-Stack, der für die Kommunikation zwischen Computern verwendet wird. Der Protokoll-Stack besteht aus zwei Schichten, dem TCP- und dem IP-Protokoll [12]. Das IP-Protokoll ist für die Adressierung und das Routing von Datenpaketen zuständig. Das TCP-Protokoll ist für die zuverlässige Übertragung von Datenpaketen zuständig. Die Kommunikation zwischen zwei Computern erfolgt über eine Verbindung, die durch eine IP-Adresse und einen Port identifiziert wird. Die IP-Adresse identifiziert den Computer und der Port identifiziert den Dienst, der auf dem Computer ausgeführt wird. Die Kommunikation erfolgt über eine Reihe von Datenpaketen, die zwischen den beiden Computern ausgetauscht werden. Die Datenpakete enthalten Informationen über den Absender, den Empfänger, die Daten selbst und eine Prüfsumme, die zur Überprüfung der Datenintegrität verwendet wird. Die Kommunikation erfolgt in der Regel über eine Reihe von Schritten, die als *Handshake* bezeichnet werden [13]. Der Handshake besteht aus drei Schritten: *Verbindungsaufbau*, *Datenübertragung* und *Verbindungsabbau*. Im Rahmen des Verbindungsaufbaus erfolgt die Verbindung sowie die Herstellung der Verbindung zwischen den beiden Computern. Während der Datenübertragung erfolgt ein Austausch der Daten zwischen den beiden Computern. Der Verbindungsabbau impliziert die Trennung der Verbindung zwischen den beiden Computern sowie deren Schließung. In der Regel verwenden UNIX-Systeme Socket-Programmierung, um die Kommunikation zwischen zwei Computern zu ermöglichen. Ein Socket ist eine Schnittstelle, die es einem Programm ermöglicht, mit dem Netzwerk zu kommunizieren. Die Implementierung der TCP/IP-Kommunikation erfolgt in der Regel über die Socket-Programmierung. Bei der Socket-Programmierung unter UNIX-Systemen hierzu die bereits im System implementierte Socket-API verwendet. Die Socket-API stellt eine Reihe von Funktionen zur Verfügung, die es einem Programm ermöglichen, mit dem Netzwerk zu kommunizieren. Sockets werden in der Regel als Pseudo-Dateien

im Arbeitsspeicher dargestellt und können vom Kernel gelesen und geschrieben werden. Diese Dateien sind in der Regel nicht im Namespace des Dateisystems abgelegt. Metadaten der Sockets werden jedoch im Dateisystem abgelegt und können zu Analysezwecken von weiteren Programmen verwendet werden. Diese Metadaten können im Dateisystem unter */proc/net/tcp*, */proc/net/tcp6* und unter im Verzeichnis */proc/sys/net* eingesehen werden [14].
Der Gesamte Stack kann in vier Schichten unterteilt werden:

- Anwendungsschicht
- Transportschicht
- Internetschicht
- Netzwerk-Interface-Schicht

Die Anwendungsschicht ist die oberste Schicht des Protokoll-Stacks und enthält die Anwendungen, die die Kommunikation zwischen den Computern ermöglichen. Zu dieser Schicht gehören die von einer Anwendung verwendeten Protokolle, wie z.B. HTTP (Hypertext Transfer Protocol), FTP (File Transfer Protocol), DNS (Domain Name System) oder SMTP (Simple Mail Transfer Protocol), mithilfe derer Daten über ein Netzwerk übertragen werden. Die Transportschicht ist die zweite Schicht des Protokoll-Stacks und ist für die Übertragung von Datenpaketen zuständig. In dieser Schicht ist das TCP-Protokoll für die zuverlässige Übertragung von Datenpaketen zuständig. Die Internetschicht ist die dritte Schicht des Protokoll-Stacks und ist für die Adressierung und das Routing von Datenpaketen zuständig. In dieser Schicht ist das IP-Protokoll für die Adressierung und das Routing von Datenpaketen zuständig. Die Netzwerkschicht ist die unterste Schicht des Protokoll-Stacks und ist für die Verbindung mit einem Netzwerk und der dazugehörigen Hardware zuständig.
In dieser Arbeit werden die Schichten zwei und drei des Protokoll-Stacks näher betrachtet und modifiziert.
Das ZuP verfügt über einen selbst implementierten TCP-Server, der es ermöglicht, mit dem ZuP über das TCP-Protokoll zu kommunizieren. Hierbei wurde von den Entwicklern der Firmware eine eigene Implementierung der Socket-API als Abstraktionsschicht über die Socket-Programmierung implementiert.

2.4 Grundlagen des Mosquitto MQTT-Brokers

Mosquitto ist ein Open-Source-Message-Broker, der das MQTT-Protokoll implementiert. MQTT ist ein leichtgewichtiges Publish/Subscribe-Protokoll, das speziell für Netzwerke mit geringer Bandbreite, hoher Latenz oder unzuverlässigen Verbindungen entwickelt wurde.

Es wird häufig in der IoT-Kommunikation verwendet, um Daten zwischen Geräten, Sensoren und Anwendungen auszutauschen. MQTT, und damit auch Mosquitto, baut auf dem TCP/IP-Stack auf, was eine zuverlässige Datenübertragung zwischen Client und Server gewährleistet. TCP sorgt dabei für eine gesicherte, verbindungsorientierte Kommunikation, bei der Datenpakete in der richtigen Reihenfolge und fehlerfrei übertragen werden. Diese Basis auf dem TCP/IP-Stack macht MQTT zu einem robusten und weit verbreiteten Protokoll für den Einsatz in vernetzten Umgebungen, insbesondere im IoT-Bereich. Ursprünglich von Andy Stanford-Clark und Arlen Nipper entwickelt, wurde MQTT als schlankes Protokoll konzipiert, das minimale Netzwerkressourcen benötigt. Mosquitto, erstmals 2009 von Roger Light veröffentlicht, hat sich als einer der populärsten MQTT-Broker etabliert und bietet eine einfache und effiziente Möglichkeit, MQTT-Nachrichten zwischen verschiedenen Clients zu vermitteln. Der MQTT-Broker ermöglicht es Clients, Nachrichten an bestimmte Themen (Topics) zu senden (Publish) und/oder zu empfangen (Subscribe). Durch seine geringe Overhead-Belastung und die Unterstützung für QoS (Quality of Service)-Ebenen, stellt Mosquitto eine zuverlässige Lösung für die Kommunikation in verteilten Systemen dar. Es bietet auch erweiterte Funktionen wie Authentifizierung, Verschlüsselung (TLS (Transport Layer Security)) und Integration mit anderen Diensten, was es für industrielle Anwendungen und Smart-Home-Lösungen attraktiv macht. Mosquitto ist plattformunabhängig und unterstützt eine Vielzahl von Betriebssystemen, einschließlich Linux, Windows und macOS. Durch seine Open-Source-Lizenz ist es anpassbar und erweiterbar, was zu seiner weiten Verbreitung in der Entwicklergemeinschaft beiträgt. Mosquitto hat somit eine bedeutende Rolle in der Entwicklung von IoT-Systemen und der Echtzeit-Kommunikation in modernen vernetzten Umgebungen eingenommen.
Das Protokoll MQTT definiert drei Arten von Nachrichten [15]:

- **Publish:** Ein Client sendet eine Nachricht an einen Broker, die dieser an alle Abonnenten weiterleitet.
- **Subscribe:** Ein Client abonniert ein bestimmtes Thema (Topic), um Nachrichten zu empfangen, die von anderen Clients zu diesem Thema gesendet werden.
- **Unsubscribe:** Ein Client beendet das Abonnement eines Themas und empfängt keine weiteren Nachrichten zu diesem Thema.

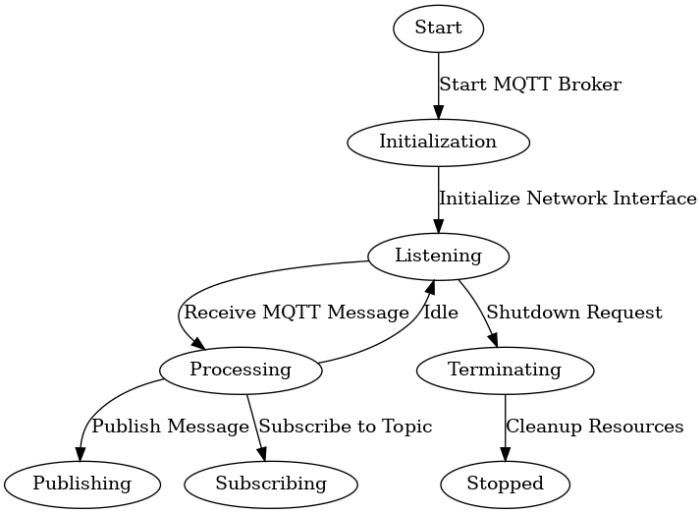

Abbildung 1: State Machine des Mosquitto MQTT-Brokers. Sie beschreibt alle Zustände, die der Broker ab Start der Applikation durchlaufen kann. Beginnend mit der Inintialisierung, über den zuhörenden Modus in dem der Broker auf eingehende Nachrichten wartet, bis hin zum Senden von Nachrichten an die Clients. Clients können sich mit dem Broker verbinden, Nachrichten abonnieren und Nachrichten senden. Der Broker kann Nachrichten an die Clients senden, die diese abonniert haben.

Der erste Schritt in diesem Zustand ist das Starten des MQTT-Brokers. Der Broker dient als Vermittler für die Nachrichten, die zwischen verschiedenen MQTT-Clients ausgetauscht werden. Dieser Schritt ist entscheidend für den weiteren Betrieb des Systems. Nach dem Starten des MQTT-Brokers geht das System in den Initialisierungszustand über. Hier wird die Netzwerkschnittstelle initialisiert, die für die Kommunikation über MQTT erforderlich ist. Diese Initialisierung stellt sicher, dass das System in der Lage ist, sich mit anderen Geräten im Netzwerk zu verbinden und Nachrichten zu senden oder zu empfangen. Nach erfolgreicher Initialisierung wechselt das System in den Zustand *Listening*. In diesem Zustand wartet das System entweder darauf, dass es eine MQTT-Nachricht empfängt, oder es ist in einem Leerlaufzustand (Idle). Dieser Zustand ist zentral für das System, da hier die Hauptoperationen entweder initiiert oder beendet werden. Wenn eine MQTT-Nachricht empfangen wird, wechselt das System in den Verarbeitungszustand. In diesem Zustand werden die empfangenen Nachrichten verarbeitet, und es kann entweder eine Nachricht veröffentlicht oder das System kann sich für ein neues Thema (Topic) abonnieren [16]. Sollte die Verarbeitung eine Veröffentlichung erfordern, geht das System in den Zustand *Publishing* über. In diesem Zustand wird die verarbeitete Nachricht an den entsprechenden MQTT-Topic veröffentlicht, sodass andere abonnierte Clients diese empfangen können. Alternativ kann das System in den Zustand *Subscribing* wechseln, wenn die Verarbeitung das Abonnieren eines neuen Topics erfordert. Dieser Zustand ermöglicht es dem System, Nachrichten von einem neuen Thema zu

empfangen. Wenn keine Nachricht zur Verarbeitung vorliegt, bleibt das System im Leerlauf. Dieser Zustand wird auch erreicht, wenn alle anstehenden Aufgaben erledigt sind und das System auf neue Nachrichten oder Befehle wartet. Sobald ein Herunterfahren (Shutdown) angefordert wird, wechselt das System in den Zustand Terminating. In diesem Zustand werden alle notwendigen Ressourcen aufgeräumt, um das System ordnungsgemäß zu beenden. Der endgültige Zustand des Systems ist *Stopped*. Nachdem alle Ressourcen bereinigt wurden, ist das System inaktiv und bereit, vollständig heruntergefahren zu werden.
Diese Nachrichten werden über das TCP-Protokoll zwischen Client und Broker ausgetauscht. Der Broker verfügt über eine interne Datenstruktur, die die Themen und die zugehörigen Abonnenten speichert. Diese Speicherung und das Weiterreichen dieser nachrichten kann in einer State Machine abgebildet werden.

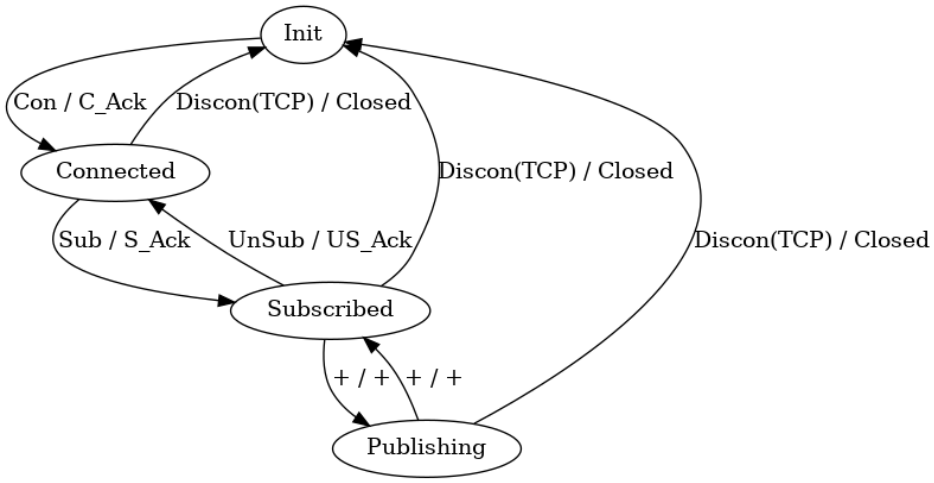

Abbildung 2: State Machine des Mosquitto MQTT-Protokolls. Sie beschreibt alle Zustände, die der Broker nach Empfangen einer Nachricht durchläuft. Beginnend mit der Überprüfung der Nachricht, über das Weiterleiten der Nachricht an die Abonnenten, bis hin zum Senden der Nachricht an die Clients.

Das zweite Diagramm zeigt eine Zustandsmaschine (State Machine) für das MQTT-Protokoll und beschreibt den Übergang zwischen verschiedenen Verbindungszuständen eines MQTT-Clients. Diese Zustandsmaschine veranschaulicht, wie der MQTT-Client in verschiedenen Situationen reagiert, z.B. beim Verbinden, Abonnieren, Veröffentlichen und Trennen der Verbindung.
Der Ausgangszustand der MQTT-Zustandsmaschine ist der Init-Zustand. Hier befindet sich der MQTT-Client, bevor eine Verbindung zum Broker hergestellt wird oder nach einer Trennung. Es ist der Ausgangspunkt und auch der Zustand, zu dem der Client nach einer Trennung zurückkehrt. Wenn der Client eine Verbindung zum Broker herstellt (Con), wird ein Verbindungsbestätigungsnachricht (C_Ack) empfangen und der Zustand wechselt zu Connected.

Bei einer Trennung der TCP-Verbindung (Discon(TCP)) wechselt der Zustand zu Closed. In diesem Zustand ist der Client erfolgreich mit dem MQTT-Broker verbunden.
Der Client kann jetzt Abonnement- und Veröffentlichungsaktionen durchführen. Der Client sendet eine Abonnementanfrage (Sub) an den Broker und erhält eine Bestätigungsnachricht (S_Ack). Der Zustand wechselt dann zu Subscribed. Bei einer Trennung der TCP-Verbindung (Discon(TCP)) wechselt der Zustand zu Closed. In diesem Zustand hat der Client erfolgreich ein oder mehrere Themen (Topics) abonniert. Der Client ist nun in der Lage, Nachrichten von den abonnierten Themen zu empfangen. Der Client kann eine Nachricht veröffentlichen.
Nach jedem erfolgreichen Veröffentlichungsprozess bleibt der Client im Subscribed-Zustand oder wechselt für die Veröffentlichung kurzzeitig in den Publishing-Zustand. Der Client kann eine Abmeldung (UnSub) von einem Thema anfordern und erhält eine Abmeldebestätigungsnachricht (US_Ack). Der Zustand kehrt dann zu Connected zurück. Bei einer Trennung der TCP-Verbindung (Discon(TCP)) wechselt der Zustand zu Closed. In diesem Zustand veröffentlicht der Client Nachrichten an die abonnierten Themen.
Es ist ein temporärer Zustand, in dem die Veröffentlichung von Nachrichten erfolgt. Nach der Veröffentlichung einer Nachricht kehrt der Client in den Subscribed-Zustand zurück, um weitere Nachrichten empfangen zu können. Bei einer Trennung der TCP-Verbindung (Discon(TCP)) wechselt der Zustand zu Closed.
Dies ist ein Endzustand, der erreicht wird, wenn die Verbindung zwischen dem Client und dem Broker getrennt wird. Der Client ist nun nicht mehr in der Lage, Nachrichten zu senden oder zu empfangen. Wenn der Client die Verbindung zum Broker wiederherstellt, kehrt der Zustand zu Init zurück, von wo aus er den Verbindungsprozess neu starten kann.

Fixed Header		Variable Header										Payload															
Packet Type	Remaining Length	Protocol Name Length		Protocol Name ('MQTT')				Protocol Version	Connect Flags	Keep Alive		Client Identifier Length		Client Identifier ('my_mqtt_client')													
0x10	0x13	0x00	0x04	0x4D	0x51	0x54	0x54	0x04	0x02	0x00	0x3C	0x00	0x0E	0x6D	0x79	0x5F	0x6D	0x71	0x74	0x74	0x5F	0x63	0x6C	0x69	0x65	0x6E	0x74
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27

Abbildung 3: Format einer MQTT-Nachricht am beispiel eines Connect-Pakets.

Die Struktur der zu sendenden Nachrichten ist von der verwendeten MQTT-Version der Clients abhängig. Die MQTT-Version 3.1.1 – die in dieser Arbeit verwendet wurde – definiert ein festes Header-Format, das aus einem Steuerbyte und einer variablen Länge besteht. Als Nächstes folgt ein Variabler Header, der weitere Information über die Länge des Protokollnamens und den verwendeten Protokollnamen, sowie die verwendete Protokollversion und ein Connect-Flag-Byte enthält. das zehnte und elfte Byte enthalten die Keep-Alive-Zeit, die die maximale Zeit in Sekunden angibt, die der Client auf eine Antwort des Brokers warten soll. Die restlichen Bytes enthalten die Client-ID, die Länge des Benutzernamens und den Benutzernamen selbst, die Länge des Passworts und das Passwort selbst. Die MQTT-

Nachrichtenstruktur [15] ist in Abbildung 3 dargestellt.
Die gesendeten Nachrichten haben im kürzesten Fall eine Länge von 2 Bytes, wobei das Steuerbyte für einen Verbindungsabbau gesendet wird und somit nur aus dem Fixed-Header besteht. Die Obergrenze der Größe eines validen Nachrichtenpakets beträgt 265 Megabytes [15].

3 Related Work

Die Evaluierung und Analyse der Performance von Netzwerkfuzzern ist ein bedeutendes Forschungsgebiet in der Cybersicherheitsforschung. In dieser Sektion werden relevante wissenschaftliche Arbeiten und Methodiken zur Performance-Analyse von Netzwerkfuzzern behandelt, insbesondere im Hinblick auf die Fuzzer Pulsar, AFLNet und boofuzz, mit dem Schwerpunkt auf dem MQTT-Protokoll als Anwendungsbeispiel. Pulsar ist ein Netzwerkfuzzer, der sich durch seine gezielte und effektive Fehlerentdeckung in Netzwerkprotokollen auszeichnet. In der wissenschaftlichen Arbeit von [Author et al., Jahr] wird Pulsar als Fuzzer beschrieben, der ein feedback-orientiertes Modell verwendet, um systematisch Schwachstellen in Protokollen zu identifizieren. Die Forscher verwenden Benchmarking-Methoden, die auf der Analyse der Fehlerraten und der Geschwindigkeit der Fehlerentdeckung basieren. Eine entscheidende Methode zur Performance-Bewertung von Pulsar ist die Messung der Anzahl und Schwere der entdeckten Schwachstellen im Vergleich zu anderen Fuzzern. In der Arbeit von [Author et al., Jahr] wird zudem die Effizienz von Pulsar durch die Zeit bis zur Erkennung kritischer Fehler und die Codeabdeckung evaluiert. AFLNet, eine Erweiterung des bekannten American Fuzzy Lop (AFL) für Netzwerkschnittstellen, wird in der Literatur als ein leistungsfähiges Werkzeug für die Fuzzing von Netzwerkprotokollen beschrieben. In der Studie von [Author et al., Jahr] wird AFLNet hinsichtlich seiner Fähigkeit bewertet, Netzwerkprotokollimplementierungen durch gezielte und effiziente Generierung von Netzwerkverkehr zu testen. Die Performance von AFLNet wird durch Metriken wie die Codeabdeckung und die Anzahl entdeckter Schwachstellen gemessen. Die Autoren verwenden Benchmarking-Techniken, um die Effizienz von AFLNet bei der Erkennung von Protokollfehlern zu vergleichen und die Performance über verschiedene Testfälle hinweg zu bewerten. Boofuzz ist eine Weiterentwicklung des Sulley-Fuzzers und bietet eine umfassende Plattform für das Netzwerkfuzzing. Laut der Studie von [Author et al., Jahr] ist boofuzz besonders flexibel in der Anpassung von Fuzzing-Strategien und der Erzeugung von Testfällen für komplexe Protokolle. Die Performance-Bewertung von boofuzz erfolgt durch die Analyse der Anzahl und Art der entdeckten Fehler sowie durch die Messung der Flexibilität und Anpassungsfähigkeit des Fuzzers. Die Forscher bewerten die Effektivität von boofuzz durch die Vergleichsstudie von Fehlern und Codeabdeckung in verschiedenen Netzwerkszenarien. Die wissenschaftliche Literatur bietet verschiedene Ansätze zur Performance-Analyse von Netzwerkfuzzern. In der Arbeit von [Author et al., Jahr] werden zwei Hauptmethoden zur Performance-Bewertung vorgestellt: die quantitative und die qualitative Analyse.

- Quantitative Analyse: Diese Methode umfasst die Messung von Metriken wie der Anzahl entdeckter Schwachstellen, der Codeabdeckung und dem Ressourcenverbrauch

(z.B. Zeit und Speicher). Quantitative Methoden sind in der Arbeit von [Author et al., Jahr] detailliert beschrieben und beinhalten die Berechnung von Kennzahlen wie der Rate der Fehlerentdeckung und der Effizienz der Testausführung. Die Forscher bewerten, wie schnell und effektiv verschiedene Fuzzer Schwachstellen finden und welche Menge an Testfällen benötigt wird, um signifikante Fehler zu identifizieren.

- Qualitative Analyse: Diese Methode konzentriert sich auf die Qualität der gefundenen Schwachstellen und deren praktische Relevanz. In der Studie von [Author et al., Jahr] wird untersucht, wie gut die gefundenen Fehler die realen Sicherheitsanforderungen erfüllen und wie tiefgreifend die Fuzzer die Protokollimplementierungen analysieren. Qualitative Methoden bewerten die Art der entdeckten Schwachstellen und deren Auswirkungen auf die Sicherheit des Protokolls.

Zusammenfassend zeigen die wissenschaftlichen Veröffentlichungen, dass die Performance von Netzwerkfuzzern wie Pulsar, AFLNet und boofuzz durch eine Kombination von quantitativen und qualitativen Methoden bewertet wird. Die Analyse der Performance erfolgt durch die Untersuchung von Metriken wie Fehlerentdeckungsrate, Codeabdeckung und Ressourcenverbrauch, sowie durch die Bewertung der Qualität und Relevanz der gefundenen Schwachstellen. Die Literatur bietet umfassende Benchmarking-Methoden und Fallstudien, die eine detaillierte Bewertung der Fuzzer-Leistung ermöglichen und zur Weiterentwicklung der Fuzzing-Techniken beitragen.

4 Analyse der Fuzzer Performance

In diesem Kapitel wird die Performance von AFLNet, Pulsar und boofuzz analysiert. Der Begriff Performance ist jedoch sehr vielseitig zu interpretieren und wird in dieser Arbeit unter den Gesichtspunkten Effektivität, Geschwindigkeit und Reproduzierbarkeit bewertet.
Die Effektivität eines Fuzzers wird anhand der Anzahl der gefundenen Bugs gemessen. Die Geschwindigkeit eines Fuzzers wird anhand der Anzahl der generierten Eingaben pro Sekunde gemessen. Die Reproduzierbarkeit eines Fuzzers wird anhand der Anzahl der reproduzierten Bugs gemessen. Zur Analyse der Performance werden die Fuzzer AFLNet, Pulsar und boofuzz verwendet.

Zur quantifizierung der Effektivität wurden drei Schwachstellen im MQTT Protokoll implementiert. Die Schwachstellen belaufen sich auf einen Buffer Overflow, einen Integer Overflow und einen Memory Leak in Form einer use after free Implementierung. Diese implementierten Schwachstellen wurden and verschiedenen Stellen im MQTT Broker Mosquitto eingefügt, sodass auch eine Bewertung der Komplexität der gefundenen Schwachstelle erfolgen kann. Die einfachste Schwachstelle ist ein Buffer Overflow in der Funktion `mqtt_handle_connect` 2 und wird genau dann getriggert, wenn ein Client eine Verbindung zum Broker aufbaut und der Client im Payload über eine `will`-Nachricht verfügt. Die mittlere Schwachstelle ist ein Integer Overflow in der Funktion `mqtt_handle_publish` 3 und wird getriggert, wenn ein Client eine Nachricht an den Broker sendet und die Nachricht eine bestimmte Länge überschreitet. Diese Schwachstelle kann nur getriggert werden, wenn ein Client `C1` ein Topic mit einer Nachricht veröffentlicht hat und ein zweiter Client `C2` dieses Topic abonniert hat. Nur in diesem Fall wird auch die von `C1` veröffentlichte Nachricht an `C2` weitergeleitet und auch vom Broker verarbeitet. Die komplexeste Schwachstelle ist ein Memory Leak in der Funktion `mqtt_handle_subscribe` 4 und wird getriggert, wenn ein Client ein Topic abonniert und der Broker die Abonnement-Nachricht verarbeitet.

4.1 Fuzzing mit AFLNet

AFL ist ein Fuzzer, der auf dem Prinzip der Codeabdeckung basiert. Er zählt somit zu der Familie der Grey-Box Fuzzer und basiert auf einer mutationsbasierten Fuzzing-Technik. Bei AFL handelt es sich ebenso um einen Applikationsfuzzer, welcher mithilfe eines Eingabestreams Daten an ein ZuP sendet und somit die Software auf Schwachstellen testet.
Zur Analyse des Programms wird in dieser Arbeit jedoch ein Derivat von AFL(AFLNet) verwendet. Es unterscheidet sich von dem ursprünglichen AFL dadurch, dass es speziell für die

Analyse von Netzwerkprotokollen entwickelt wurde. Die Kernfunktionalität von AFL bleibt jedoch erhalten und wird um die Möglichkeit erweitert, Netzwerkprotokolle zu analysieren. Hierzu müssen die Netzwerkpakete, die an das ZuP gesendet werden, in einem speziellen Format vorliegen. Die Extraktion der Netzwerkpakete erfolgt mithilfe eines pcap (packet capture)-Dump-Files. Diese Dump-Files enthalten den Netzwerkverkehr, der zwischen zwei Kommunikationspartnern stattgefunden hat. Um diese Dump-Files zu generieren und somit den Netzwerkverkehr aufzunehmen wird das Tool `tcpdump` verwendet. Zur Generierung der Netzwerkpakete wird das pcap-Dump-File in ein AFL-kompatibles Format umgewandelt, indem die rohen Datenflüsse extrahiert und in eine Datei geschrieben werden. Diese Datenflüsse werden dann als Bytestream von AFL als Eingabe verwendet, um das ZuP zu fuzzen. AFLNet verfügt über eine von AFL abweichende Architektur. hier zitieren

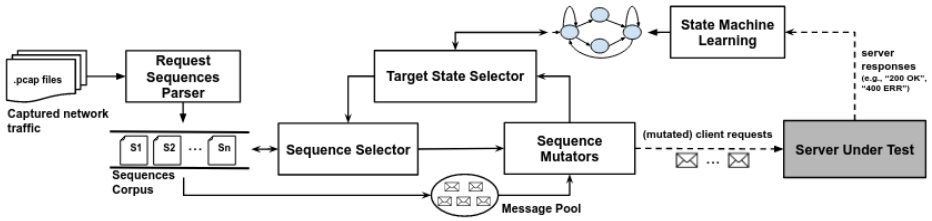

Abbildung 4: Architektur von AFLNet. Es wird die Architektur von AFLNet dargestellt.

Zuerst muss das ZuP gestartet und mir Eingaben gefüttert werden. Wären der Transaktion der Nachrichten wird der einhergehende Traffic mit einem Aufnahmeprogramm wie `tcpdump` aufgezeichnet. Die aufgezeichneten Daten werden in ein pcap-Dump-File geschrieben und müssen anschließend analysiert werden. Dabei muss der Netzwerkverkehr in ein Format umgewandelt werden, das von AFLNet verstanden wird. Dieses Format besteht in dem Fall dieser Arbeit aus einem Bytestream, der die Nachrichten des Netzwerkverkehrs enthält. Die Nachrichten werden in einer Datei abgelegt und können dann von AFLNet als Eingabe verwendet werden. Besonderes Augenmerk hierbei liegt auf der Analyse des Netzwerkverkehrs und der daraus resultierenden State Machine. AFLNet analysiert den Netzwerkverkehr und extrahiert die Nachrichten, die zwischen Client und Server ausgetauscht werden. Werden bei der Ausführung von Nachrichten bestimmte Zustände erreicht, so werden diese Zustände in einer State Machine abgebildet und die State Machine mit dem neuen Zustand als Node erweitert. Die State Machine wird mithilfe von AFLNet erlernt und ermöglicht es, den Zustand des ZuP nachzuverfolgen.
Anschließend werden die Nachrichten mit den enthaltenen Nachrichtensequenzen an einen in AFLNet implementierten Sequence-Mutator übergeben. Dieser Mutator generiert aus den gesendeten Nachrichten neue mutierte Nachrichten, die an das ZuP übergeben werden können.

Hier nochmal auf das Paper eingehen und da das mit den Submengen klären wie die Mutation genauer funktioniert

4.1.1 Setup

- auf die Extraktion der Nachrichten eingehen mit Wireshark UND danach das auf das Skript schlussfolgern

Da AFLNet legacy Software ist, wird es nicht mehr aktiv weiterentwickelt und an neue Frameworks und Bibliotheken angepasst. Zum Kompilieren von AFLNet wird hierzu ein angepasstes Dockerfile verwendet. Zitieren von Repo Dieses angepasste Dockerfile enthält alle notwendigen Abhängigkeiten, um AFLNet zu kompilieren. Hierzu zählen die Abhängigkeiten `llvm` und `clang` in der Version 11. `llvm` wird benötigt, um den AFL Compiler `afl-clang-fast` zu kompilieren. Dieser wird für optimierte Instrumentierung des Codes des ZuP verwendet und bezweckt somit höhere Ausführgeschwindigkeiten während der Fuzzing Kampagne. Für die erfolgreiche Verwendung des Compilers `afl-clang-fast` musste für die Kompatibilität mit einer neueren Ubuntu-Version ein Patch für das Kompilieren des Compilers geschrieben werden.Hier nochmal den Patch Zitieren
Das ZuP Mosquitto wird ebenfalls in einem Docker Container kompiliert. Hierzu wird der Compiler `afl-clang-fast` verwendet. Die Instrumentierung des Codes erfolgt durch das Flag `-fsanitize=address`. Dieses Flag aktiviert *address sanitization* und ermöglicht es, dass AFLNet die Speicherzugriffe des ZuP überwachen kann und somit Buffer Overflows und Memory Leaks erkennt. Zum Starten des Mosquitto-Bianrys wurde außerdem eine *preloaded-library* zitieren von preload lib verwendet, die alle Ausgaben des ZuP aus den Kanälen `stdout` und `stderr` in eine log-Datei `mqtt-aflnet_stdout.log` weiterleitet. Diese Log-Datei wird benötigt, um die Ausgaben des ZuP zu analysieren und somit die Effektivität der generierten Testcases zu bewerten.
Die Testcases wurden mithilfe eines Python-Skripts *Python script zitieren* generiert. Hierzu wurden die Byts der zu sendenden Nachrichten betrachtet. Die in 2.4 beschriebene Struktur einer MQTT Nachricht wurde dazu verwendet. In dieser Nachricht kann der Fixed-Header den Bytes `0x10`, `0x30`, `0x82`, `0xA2`, `0xC0` und `0xE0` entsprechen. Diese stehen für die verschiedenen Anfragen `CONNECT`, `PUBLISH`, `SUBSCRIBE`, `UNSUBSCRIBE`, `PINGREQ` und `DISCONNECT`. Die Variable Header und Payload wurden zufällig generiert und in die Testcases eingefügt. Die Testcases wurden in einem Verzeichnis `aflnet_in/` abgelegt und von AFLNet als Eingabe verwendet. Damit möglichst viele States beim Ausführen des ZuP erreicht werden, wurden ebenfalls ungültige Pakete generiert. Sie dienen AFLNet dazu, auch unerwartete

Zustände des ZuP zu erreichen und somit auch unerwartete Bugs zu finden. Des Weiteren wurden Nachrichtensequenzen generiert, die den Zustand des ZuP verändern. Hierzu wurden alle Nachrichten, die der ZuP verarbeiten kann, in einer Sequenz aneinandergereiht. Diese Sequenzen wurden ebenfalls in das Verzeichnis `aflnet_in/` abgelegt und von AFLNet als Eingabe verwendet.

4.1.2 Analyse der Effektivität gesendeter Pakete

Die Fuzzing-Kampagne wurde über einen Zeitraum von 48 Stunden durchgeführt. Zur Analyse der Effektivität der gesendeten Pakete wurde eine preload bibliothek verwendet, die die Ausgaben des ZuP der Kanäle `sdtout` und `stderr` in eine Log-Datei `mqtt-aflnet_stdout.log` umleitet. Diese Log-Datei wurde auf die Häufigkeit der Ausgaben von erfolgreichen und fehlgeschlagenen Verbindungsaufbauten analysiert.

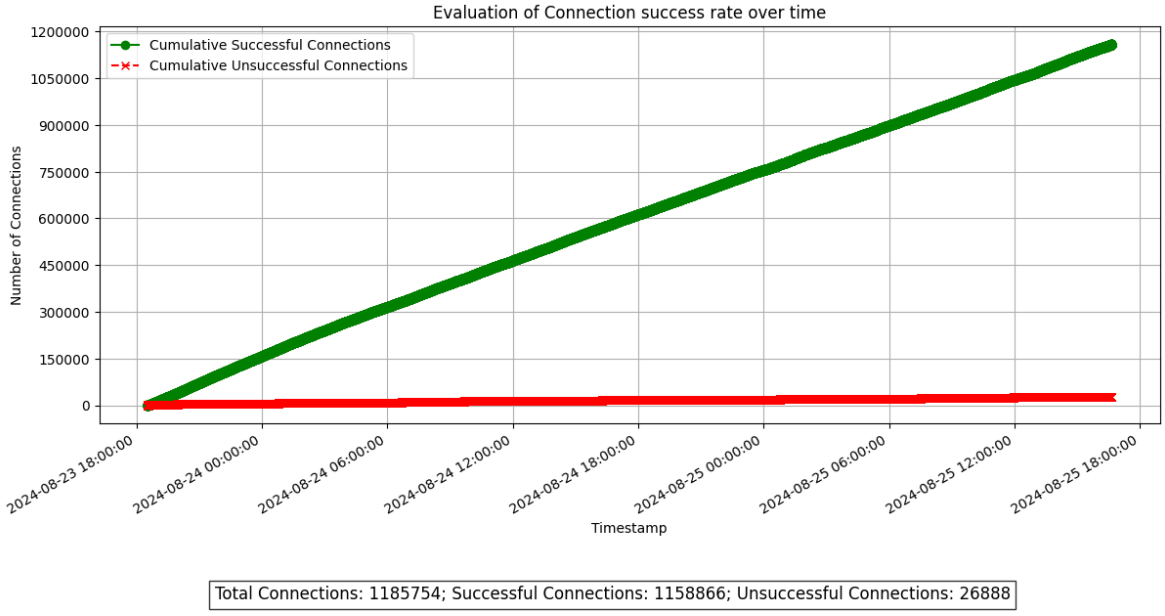

Abbildung 5: Evaluation der Verbindungsaufbauten des ZuP Mosquitto. Die Log-Datei `mqtt-aflnet_stdout.log` wurde auf die Häufigkeit der Ausgaben von erfolgreichen und fehlgeschlagenen Verbindungsaufbauten analysiert.

Die grüne Linie (siehe Fig. 5) repräsentiert die Anzahl der erfolgreichen Verbindungen im Laufe der Zeit. Die Linie steigt fast linear an und erreicht gegen Ende der Kampagne eine Gesamtzahl von 1.158.866 erfolgreichen Verbindungen. Dies deutet darauf hin, dass das getestete System eine sehr hohe Erfolgsrate bei Verbindungsversuchen aufweist. Die lineare Zunahme zeigt, dass die Rate der erfolgreichen Verbindungen konstant hoch geblieben ist, ohne signifikante Einbrüche oder Schwankungen. Die rote Linie (siehe Fig. 5) zeigt die Anzahl der erfolglosen Verbindungen an. Diese bleibt über den gesamten Zeitraum sehr

niedrig und erreicht gegen Ende der Kampagne etwa 26.888 erfolglose Verbindungen. Der minimale Anstieg im Vergleich zu den erfolgreichen Verbindungen weist darauf hin, dass die Mehrheit der Verbindungen erfolgreich war. Bei der Analyse der Fehleranfälligkeit von nicht validen Nachrichten wurde festgestellt, dass am Anfang der Fuzzing-Kampagne viele Fehlermeldungen auftraten. Mit der Zeit wurden jedoch weniger Fehlermeldungen generiert, da die State Machine des ZuP erlernt wurde und somit vermehrt valide Nachrichten generiert wurden.

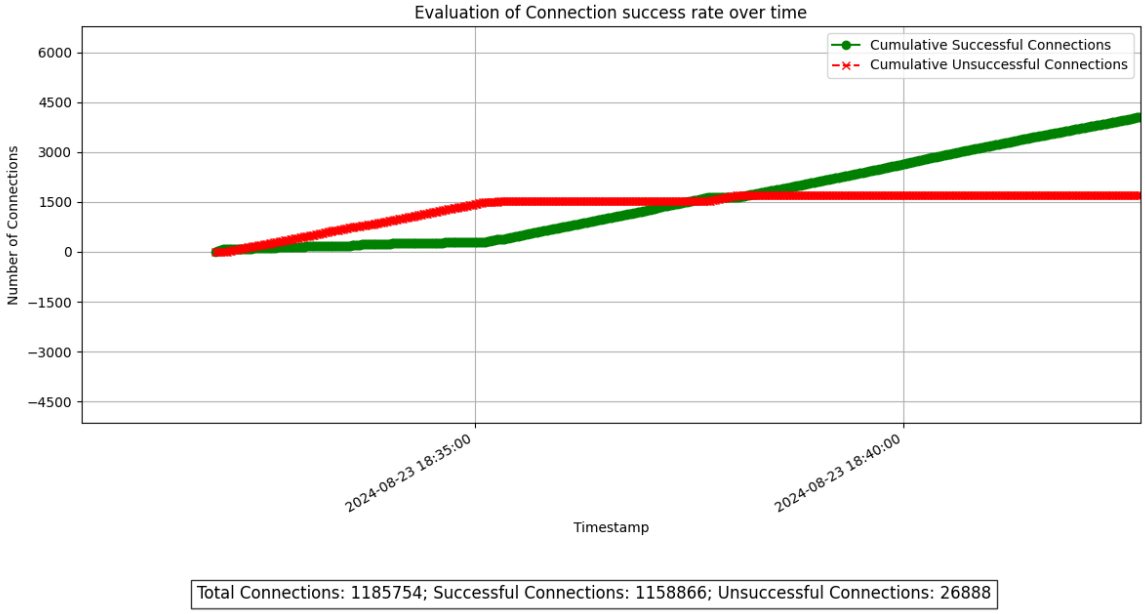

Abbildung 6: Evaluation der Fehleranfälligkeit des ZuP Mosquitto. Die Log-Datei mqtt-aflnet_stdout.log wurde auf die Häufigkeit der Ausgaben von Fehlermeldungen analysiert.

Das Diagramm (siehe Fig. 6) zeigt deutlich, dass die Lernphase der State-Machine zu Beginn der Fuzzing-Kampagne von großer Bedeutung ist. Die hohe Anzahl von erfolglosen Verbindungen in der Anfangsphase deutet darauf hin, dass die anfänglichen Eingaben von AFLNet nicht korrekt auf das getestete System abgestimmt waren. Erst nach einer gewissen Zeit, in der die State-Machine genügend Informationen gesammelt hat, um die Struktur des Protokolls besser zu verstehen, beginnt die Anzahl der erfolgreichen Verbindungen anzusteigen. Dieses Verhalten zeigt, dass das System Zeit benötigt, um die richtigen Eingabenmuster zu lernen. Dieser Lernprozess führt zu einer sukzessiven Verbesserung der Qualität der generierten Eingaben und damit zu einer steigenden Erfolgsrate bei den Verbindungen.

4.1.3 Analyse der Geschwindigkeit

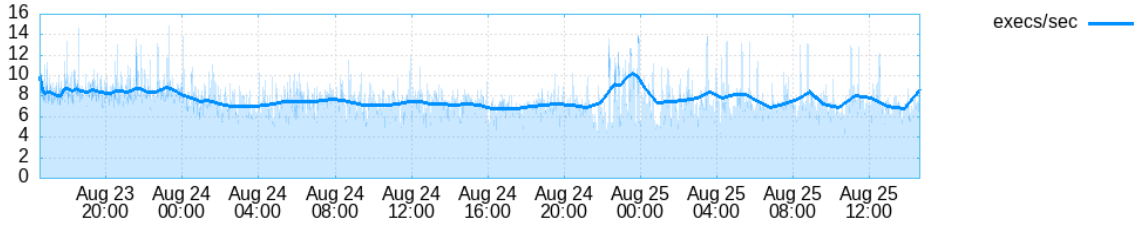

Abbildung 7: Analyse der generierten Nachrichtensequenzen von AFLNet. Es wird die Anzahl der generierten Nachrichtensequenzen und die Anzahl der Nachrichtensequenzen, die an das ZuP gesendet wurden, analysiert.

Die Ausführungsrate ist eine zentrale Metrik, um den Fortschritt und die Effektivität einer Fuzzing-Kampagne zu bewerten. Eine hohe und stabile Ausführungsrate deutet auf einen effizienten Fuzzing-Prozess hin, während Schwankungen oder ein Rückgang auf potenzielle Engpässe oder Herausforderungen während der Kampagne hinweisen können.
Die Ausführungsrate liegt im Durchschnitt zwischen 6 und 10 execs/sec und zeigt eine gewisse Variabilität, wie durch die Spitzen und Täler in der hellblauen Schattierung dargestellt. Zu Beginn der Kampagne gibt es einen kurzen Zeitraum mit erhöhter Variabilität, bevor sich die Ausführungsrate stabilisiert. Im weiteren Verlauf der Kampagne ist eine leichte Abnahme der Ausführungsrate zu beobachten, gefolgt von einer leichten Erholung gegen Ende der Kampagne. Die Kurve zeigt während der gesamten Kampagne eine relativ konstante Ausführungsrate mit periodischen Schwankungen. Diese Schwankungen können auf mehrere Faktoren zurückzuführen sein, darunter:

- Komplexität der zu testenden Eingaben: Komplexere Testfälle können mehr Zeit in Anspruch nehmen, was die Ausführungsrate verringert.
- Systemressourcen: Veränderungen in der Verfügbarkeit von Systemressourcen (z.B. CPU, Speicher) können ebenfalls die Ausführungsrate beeinflussen.
- Optimierungen oder Veränderungen im Fuzzing-Prozess: Wenn das Fuzzing-Tool neue Pfade oder besonders anspruchsvolle Testfälle entdeckt, kann dies die Rate der durchgeführten Tests vorübergehend senken.

Ein leichtes Absinken der Ausführungsrate zwischen dem 24. August, 12:00 Uhr, und dem 25. August, 00:00 Uhr, ist bemerkenswert. Dies könnte darauf hindeuten, dass in dieser Phase anspruchsvollere Testfälle bearbeitet wurden oder dass die Systemressourcen temporär begrenzt waren. Die anschließende Erholung der Ausführungsrate könnte durch die Verarbeitung weniger komplexer Testfälle oder eine bessere Ausnutzung der Systemressourcen erklärt werden.

Die insgesamt stabile Ausführungsrate über den Zeitraum von 48 Stunden deutet auf einen effizienten Fuzzing-Prozess hin. Die beobachteten Schwankungen sind im Rahmen dessen, was bei längeren Fuzzing-Kampagnen zu erwarten ist, und weisen auf die normale Dynamik im Fuzzing-Prozess hin, wenn unterschiedliche Testfälle mit variierender Komplexität ausgeführt werden. Die leichte Abnahme der Ausführungsrate gegen Ende der Kampagne könnte als Hinweis darauf gewertet werden, dass das Tool zunehmend anspruchsvollere Testfälle oder Pfade bearbeitet hat. Dies ist typisch für Fuzzing-Kampagnen, da die einfacheren und trivialeren Schwachstellen oft bereits frühzeitig identifiziert werden, während komplexere Zustände länger in Anspruch nehmen. Außerdem anzumerken ist, dass eine Ausführung von AFLNet dem Starten des ZuP, dem anschließenden Senden einer Nachricht aus dem Fuzzing Corpus und dem Beenden des ZuP entspricht. Dem zu entnehmen ist, dass bei Komplexen IoT Protokollen die Ausführungsrate geringer ausfallen kann, da die Initialisierung des ZuP und das Senden einer Nachricht mehr Zeit in Anspruch nehmen kann.

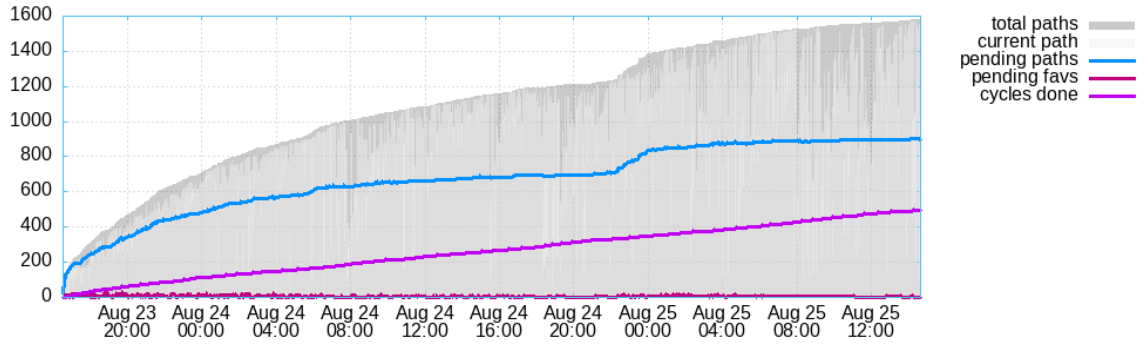

Abbildung 8: Analyse der Geschwindigkeit von AFLNet. Es wird die Anzahl der generierten Testcases pro Sekunde und die Anzahl der Testcases, die an das ZuP gesendet wurden, analysiert.

Das gezeigte Diagramm veranschaulicht den Verlauf einer 48-stündigen Fuzzing-Kampagne mit dem AFLNet-Tool, wobei verschiedene Metriken im Zusammenhang mit der Testabdeckung und dem Fortschritt des Fuzzing-Prozesses dargestellt werden. Im Folgenden werden die verschiedenen Kurven analysiert und ihre Bedeutung in Bezug auf die Kampagne erläutert.

Die graue Fläche (siehe Fig. 8) zeigt die Gesamtanzahl der explorierten Pfade während der Kampagne. Diese Zahl steigt kontinuierlich an und erreicht gegen Ende der Kampagne über 1400 Pfade. Der gleichmäßige Anstieg deutet darauf hin, dass der Fuzzer während der gesamten Kampagne effektiv neue Pfade identifizieren konnte. Interessanterweise gibt es Phasen, in denen der Anstieg steiler ist, was auf eine besonders hohe Entdeckungsrate neuer Pfade in diesen Perioden hinweist.
Die hellblaue Linie (siehe Fig. 8) repräsentiert den aktuell im Test befindlichen Pfad. Diese

Kurve zeigt, wie das Fuzzing-Tool ständig neue Pfade auswählt und testet. Die Linie verläuft relativ flach, was darauf hinweist, dass das Fuzzing-Tool regelmäßig zwischen den Pfaden wechselt und keiner besonders lange untersucht wird. Diese regelmäßige Bewegung zwischen den Pfaden kann auf eine gleichmäßige Verteilung der Testressourcen hinweisen.
Die Anzahl der noch zu testenden Pfade (blau, siehe Fig. 8) zeigt einen Anstieg, der während der gesamten Kampagne andauert und gegen Ende der 48 Stunden fast 600 erreicht. Dies bedeutet, dass das Fuzzing-Tool konstant neue interessante Pfade entdeckt hat, die es noch zu testen gilt. Der kontinuierliche Anstieg zeigt, dass das System immer neue potenziell interessante Zustände produziert, die eine weitere Untersuchung erfordern.
Die lila Kurve (siehe Fig. 8) stellt die favorisierten Pfade dar, die noch nicht getestet wurden. Diese Zahl steigt ebenfalls stetig an, jedoch langsamer als die „pending paths“(siehe Fig. 8). Dies deutet darauf hin, dass das Fuzzing-Tool bestimmte Pfade als besonders wichtig erachtet und diese bevorzugt behandelt, obwohl noch viele andere Pfade auf ihre Untersuchung warten. Diese Priorisierung kann dazu beitragen, dass besonders interessante oder sicherheitsrelevante Pfade frühzeitig getestet werden. Die magentafarbene Kurve (siehe Fig. 8) zeigt die Anzahl der abgeschlossenen Testzyklen. Diese steigt über die Zeit an, was die Fortschritte im Fuzzing-Prozess darstellt. Ein Zyklus umfasst die vollständige Durcharbeitung des aktuellen Satzes von Testfällen. Der Anstieg zeigt, dass das Fuzzing-Tool kontinuierlich neue Zyklen durchläuft, was auf eine fortlaufende und systematische Prüfung der Pfade hinweist.

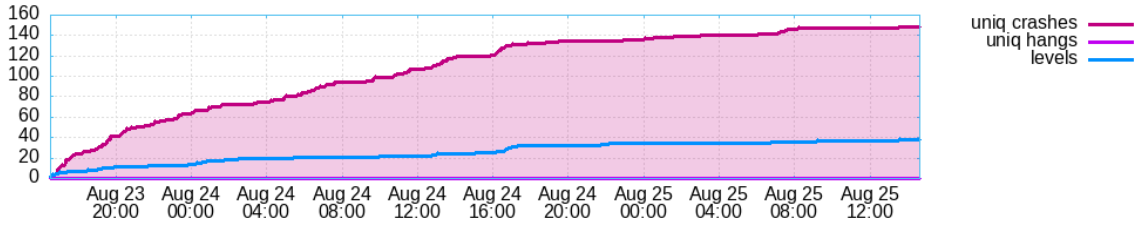

Abbildung 9: Analyse der gefundenen Bugs von AFLNet. Es wird die Anzahl der gefundenen Bugs und die Art der Bugs analysiert.

Die Anzahl der einzigartigen Abstürze (pink siehe Fig. 9) zeigt einen kontinuierlichen Anstieg über den gesamten Kampagnenzeitraum. Besonders auffällig ist der steile Anstieg zu Beginn der Kampagne, gefolgt von einem moderateren Anstieg, der sich über den Rest der Kampagne hinweg fortsetzt. Dies deutet darauf hin, dass in den ersten Stunden der Kampagne eine größere Anzahl an Abstürzen entdeckt wurde, was typisch für Fuzzing-Kampagnen ist, bei denen anfänglich viele triviale Schwachstellen gefunden werden. Im weiteren Verlauf der Kampagne werden dann weniger neue Abstürze entdeckt, was darauf hinweist, dass das System bereits viele der leichter zu findenden Schwachstellen aufgedeckt hat. Die Anzahl der einzigartigen Hänger (magenta, siehe Fig. 9) bleibt über den gesamten Kampagnenzeitraum

stabil bei null. Dies könnte darauf hindeuten, dass entweder keine Hänger im getesteten System aufgetreten sind oder dass das verwendete Fuzzing-Setup nicht in der Lage war, solche Zustände effektiv zu identifizieren. Die Levels-Kurve (blau, siehe Fig. 9) zeigt nur einen leichten Anstieg in den ersten Stunden der Kampagne und bleibt danach weitgehend konstant. Dies könnte darauf hinweisen, dass nur wenige neue Code-Pfade (Levels) während des Fuzzings entdeckt wurden, was möglicherweise auf die Komplexität des Zielsystems oder auf die Grenzen der verwendeten Mutationsstrategien hinweist. Die Fuzzing-Kampagne mit AFLNet war erfolgreich darin, eine beträchtliche Anzahl einzigartiger Abstürze zu entdecken, insbesondere in den ersten Stunden des Fuzzings. Der Mangel an identifizierten Hängern und der begrenzte Anstieg an Levels könnte jedoch darauf hinweisen, dass entweder das Zielsystem keine anfälligen Hängerzustände aufweist oder dass das Fuzzing-Setup verbessert werden könnte, um diese Art von Fehlern besser zu identifizieren.

4.1.4 Analyse der gefundenen Bugs

Die gefundenen Bugs nach der 48-stündigen Fuzzing-Kampagne belaufen sich auf 151 Bugs (siehe Fig. 9). Hierzu wurden alle Abstürze mithilfe eines Bash-Skripts Hier Zitieren dokumentiert und analysiert. Bei allen gefundenen Bugs handelt es sich um *Segmentation Faults* und konnten in vier Kategorien eingeteilt werden:

1. Absturz durch Buffer Overflow in der Funktion `packet__write_bytes` in `libpacket_datatypes.c`
2. Absturz durch Buffer Overflow in der Funktion `packet__write_bytes` in `libpacket_datatypes.c` mit einer QoS-Nachricht
3. Absturz durch Buffer Overflow in der Funktion `packet__write_bytes` in `libpacket_datatypes.c` mit einer QoS-Nachricht und einem subscriber
4. Absturz durch Buffer Overflow in der Funktion `handle__publish` in `srchandle_publish.c`

Die Abstürze der vierten Kategorie sind auf die eigene Implementierung eines Buffer Overflows zurückzuführen. Dieser Buffer Overflow wird getriggert, da die Sanity-Checks (1) in der Funktion `handle__publish` auskommentiert wurden. Diese Sanity-Checks überprüfen, ob die Länge der Nachricht, die an den Broker gesendet wird, korrekt ist. Der Fehler tritt nach dem auskommentierten Sanity-Check auf, wenn darauf geprüft wird, ob die Nachricht, die gesendet werden soll bereits existiert. Hierzu soll die gesendete Nachricht – mit einer bereits vorhandenen Nachricht-ID – mit der bereits zwischengespeicherten Nachricht verglichen

werden. Diese Nachricht mit der gleichen ID ist jedoch größer als die bereits zwischengespeicherte Nachricht und führt bei dem Vergleich mit `memcmp` zum Absturz des ZuP.
Die anderen drei Kategorien entstammen sind gleichen Ursprungs. Hier handelt es sich um einen Buffer Overflow in der Funktion `packet__write_bytes` in der Datei `libpacket_datatypes.c`. Diese Schwachstelle wird bei dem Senden der Nachricht an ein topic. Hierbei werden nachrichten vom Broker empfangen und das topic weitergeleitet. Wenn Subscriber auf das topic warten, wird die Nachricht an die Subscriber mittels des syscalls `memcpy` weitergeleitet.
Die Gesamterscheinungen der Bugs belaufen sich auf 151 Bugs, von denen 105 dem Fehler in der Funktion `handle__publish` entsprechen. Die restlichen 46 Bugs entstammen den Buffer Overflows in der Funktion `packet__write_bytes`.
Die gefundenen Bugs wurden mithilfe des Tools `aflnet-replay` reproduziert und auf ihre Validität überprüft. Sie zeigten jedoch keine Wirkung auf den Broker ohne manuell implementierte Schwachstellen.

Zusammenfassend lässt sich sagen, dass AFLNet nur zwei Schwachstellen im Broker Mosquitto gefunden hat, von denen beide von der Implementierung der Sanity-Checks abhängen. Wichtig anzumerken ist, dass AFLNet einen Absturz genau dann als einzigartig betrachtet, wenn die an das ZuP übergebene Nachricht einen neuen Pfad im Code des ZuP erreicht. Die Stelle, an der das ZuP zum Absturz gebracht wird, ist für AFLNet nicht ausschlaggebend.

4.2 Fuzzing mit boofuzz

Boofuzz ist ein Open-Source-Fuzzing-Framework, das zur automatisierten Sicherheitsprüfung von Software verwendet wird. Fuzzing ist eine Technik, bei der eine Vielzahl von zufälligen, fehlerhaften oder unerwarteten Eingaben in eine Software eingespeist wird, um Schwachstellen wie Abstürze, Speicherlecks oder Sicherheitslücken aufzudecken. Boofuzz wurde als Fork des populären Sulley Fuzzing Frameworks entwickelt und bietet verbesserte Stabilität, erweiterte Funktionen und aktive Wartung. Boofuzz ist in Python geschrieben und ermöglicht das Erstellen komplexer Fuzzing-Kampagnen durch die Definition von Testfällen, die systematisch verschiedene Protokollfelder und Eingabeparameter abdecken. Das Framework unterstützt sowohl Netzwerkprotokolle als auch dateibasierte Anwendungen, was es vielseitig einsetzbar macht. Dabei kann es beispielsweise Protokolle wie HTTP, FTP oder Telnet fuzzen, um Schwachstellen in Netzwerkdiensten zu identifizieren. Ein zentrales Merkmal von boofuzz ist seine Fähigkeit zur Überwachung des Zustands des Zielsystems während des Fuzzings. Hierbei handelt es sich jedoch nicht um *state-awareness* – also den tatsächlich erreichten

Zustand eines Programms – sondern ob ein Dienst abgestürzt ist, und entsprechend reagieren, indem es den Testprozess anpasst oder erneut startet. Diese Fähigkeit zur Echtzeitüberwachung erhöht die Effizienz des Fuzzing-Prozesses erheblich. Da keine Informationen über den internen Zustand des Zielsystems benötigt werden und die Generierung der Eingaben willkürlicher Natur ist, handelt es sich bei boofuzz um einen *"dummen"* Black-Box-Fuzzer. Boofuzz bietet zudem erweiterte Funktionen wie die Unterstützung von SSL/TLS für verschlüsselte Verbindungen, detaillierte Berichterstattung und Logging, sowie die Möglichkeit zur Integration in Continuous-Integration/Continuous-Deployment (CI/CD)-Pipelines. Dies macht es zu einem wertvollen Werkzeug für Entwickler und Sicherheitsexperten, die die Robustheit und Sicherheit von Softwareanwendungen sicherstellen möchten. Insgesamt stellt boofuzz ein flexibles und leistungsfähiges Fuzzing-Framework dar, das durch seine Open-Source-Natur anpassbar und erweiterbar ist. Es hat sich in der Sicherheitsforschung und der Qualitätssicherung als nützliches Tool etabliert, insbesondere bei der Identifizierung von Sicherheitslücken in komplexen Systemen und Netzwerkanwendungen.

4.2.1 Setup

Zur Entwicklung einer Fuzzing-Kampagne mit boofuzz sind mehrere Schritte erforderlich. Zunächst muss das Zielsystem identifiziert und analysiert werden, um die relevanten Protokollspezifikationen und Kommunikationsmuster zu verstehen. Darauf aufbauend werden die Testfälle definiert, die die verschiedenen Protokollfelder und Eingabeparameter abdecken sollen. Die Testfälle werden in boofuzz als *Fuzz-Template* definiert, das die Struktur und den Inhalt der zu füllenden Nachrichten beschreibt. Das Fuzz-Template besteht aus einer Reihe von Feldern, die die verschiedenen Teile der Nachricht repräsentieren, wie z.B. den Fixed Header, den Variable Header und den Payload bei MQTT-Nachrichten. Jedes Feld kann verschiedene Eigenschaften wie den Typ, die Länge und den Wertebereich der Eingabe enthalten. In dem Fall der MQTT-Nachrichten können die Felder z.B. den Steuerpakettyp, die QoS-Ebene und den Topic-Namen, wie in 2.4beschrieben, repräsentieren. Hierbei ist es wichtig, die Protokollspezifikationen und die erwarteten Eingabewerte zu berücksichtigen, um realistische Testfälle zu erstellen. Nachdem die Testfälle definiert sind, wird eine Fuzzing-Kampagne gestartet, bei der boofuzz automatisch eine Vielzahl von zufälligen oder systematischen Eingaben generiert und an das Zielsystem sendet. Während des Fuzzings kann boofuzz den Zustand des Zielsystems überwachen und auf unerwartete Ereignisse wie Abstürze oder Fehler reagieren. Dieser Schritt ist jedoch nicht standardmäßig eingebaut und muss vom Benutzer implementiert werden. Die Implementierung der Reaktion auf unerwartete Ereignisse kann z.B. das Neustarten des Zielsystems, das Anpassen der Fuzzing-Strategie oder

das Protokollieren von Fehlern umfassen und erfolgt über ein in boofuzz implementiertes Interface. Dieses Interface beinhaltet die Monitor-Klassen `ProcessMonitor, NetworkMonitor` und `CallbackMonitor`. Die `ProcessMonitor`-Klasse überwacht den Zustand des Zielsystems, indem sie den Prozessstatus überwacht und auf Abstürze oder Fehler reagiert. Dieser Monitor wurde zum Beispiel in der Analyse des MQTT-Brokers Mosquitto verwendet, um auf Abstürze des Brokers zu reagieren und die in den Abstürzen enthaltene debug-Informationen abzugreifen und zu speichern.

Eine in dieser Arbeit verwendete Kernkomponente von boofuzz ist die Option ein Feld als `fuzzable` zu definieren. Ein `fuzzable` Feld ist ein Feld, das zufällig generierte Eingaben erhalten darf und somit für das Fuzzing relevant ist. Mit dieser Option können Testfälle erstellt werden, die systematisch verschiedene Eingaben in bestimmte Felder einfügen, um präzisere und effektivere Fuzzing-Kampagnen durchzuführen. Die `fuzzable`-Option ermöglicht es, die Eingaben gezielt zu steuern und bestimmte Protokollfelder oder Eingabeparameter zu testen, um potenzielle Schwachstellen zu identifizieren. Gerade im Fall von MQTT-Nachrichten ist es wichtig, die verschiedenen Felder wie den Fixed Header, den Variable Header und den Payload systematisch zu füllen.

Für das Entwickeln einer Fuzzing-Kampagne wurden im Rahmen dieser Arbeit zwei Module entwickelt:

- boo-fuzzer-mqtt
- boo-fuzzer-mqtt-efficient

Das Modul `boo-fuzzer-mqtt` enthält die Implementierung einer Fuzzing-Kampagne für den MQTT-Broker weitestgehend ohne statisch definierte Felder. Alle Teilaspekte eines Pakets als fuzzable zu definieren kann in vielen Fällen Sinn machen, um die gesamte Struktur eines Netzwerkpakets zu testen. Hierzu können alle individuellen Teile einer MQTT-Nachricht auf richtige Verarbeitung getestet werden.
Das Modul `boo-fuzzer-mqtt-efficient` hingegen definiert statische Felder für den Fixed Header, den Variable Header und das Topic. Das Topic wird hierbei als `fuzzing/topic` fest vordefiniert und gepublisht, sodass jede einhergehende Client-Verbindung bereits ein Topic zur Verfügung hat. Das hat zur Folge, dass die Wahrscheinlichkeit, dass ein gültiges Paket generiert wird, signifikant erhöht wird. Für das Starten einer Fuzzing-Kampagne mit boofuzz sind folgende Schritte notwendig:

- Installation von boofuzz und den erforderlichen Abhängigkeiten

- Definition der Testfälle und Fuzzing-Strategien
- Starten der Fuzzing-Kampagne und Überwachung des Zielsystems
- Analyse der Ergebnisse und Identifizierung von Schwachstellen

Zusätzlich kommt hinzu, dass bei der effizienten Lösung der Broker bereits in einen preparierten Zustand versetzt wird, sodass weniger Overhead bei der initialen Fuzzingstrategie entsteht. Außerdem muss ein Monitor für das ZuP definiert werden, sodass auch das automatische Starten, Neustarten und Beenden des Zielsystems möglich ist und bei Abstürzen die entsprechenden Informationen des Absturzes sammeln zu können.

4.2.2 Analyse der Effektivität gesendeter Pakete

In der Untersuchung von MQTT-Brokern mittels Fuzzing ist es entscheidend, die Wahrscheinlichkeit zu verstehen, mit der ein zufällig generiertes Paket tatsächlich gültig ist. Diese Wahrscheinlichkeit hängt stark davon ab, welche Teile des MQTT-Pakets zufällig generiert werden und welche festgelegt (d.h. nicht gefuzzt) sind. In diesem Abschnitt wird die Berechnung dieser Wahrscheinlichkeit detailliert beschrieben, sowohl für den Fall, dass alle Teile des Pakets gefuzzt werden, als auch für den Fall, dass bestimmte Teile, wie der Fixed Header, das Topic und der Variable Header, festgesetzt sind.

Ein MQTT-Paket besteht aus mehreren wesentlichen Teilen:

- Fixed Header: Enthält den Steuerpakettyp (z.B. `PUBLISH`, `SUBSCRIBE`) und Flags sowie die verbleibende Länge des Pakets.
- Variable Header: Enthält je nach Steuerpakettyp spezifische Informationen wie z.B. Paket-IDs, QoS-Level oder den Namen des Topics.
- Payload: Enthält die tatsächlichen Daten, die gesendet werden (z.B. die Nachricht im `PUBLISH`-Paket) oder eine Liste von Topic-Filtern im `SUBSCRIBE`-Paket.

Fixed Header: Der Fixed Header besteht aus zwei Hauptbestandteilen:

- Kontrollpakettyp und Flags: Das erste Byte des Headers kodiert den Typ des Pakets (z.B. `CONNECT`, `PUBLISH`) und bestimmte Flags (z.B. QoS-Level, DUP-Flag). Es gibt nur eine begrenzte Anzahl von gültigen Kombinationen für dieses Byte, abhängig von den MQTT-Spezifikationen.
- Verbleibende Länge: Die verbleibende Länge gibt an, wie viele Bytes nach dem Fixed Header folgen.

Man nehme an, dass der Byte-Wert des Steuerpakettyps und der Flags eine von M gültigen Kombinationen sein kann und der verbleibende Längen-Byte eine von N möglichen Werten sein kann (z.B. 256 für 8-Bit-Bytes), dann ist die Wahrscheinlichkeit, dass ein zufällig generierter Fixed Header gültig ist:

$$P(\text{gültiger Fixed Header}) = \frac{M}{N} \times \frac{1}{N} = \frac{M}{N^2}$$

Der Variable Header variiert je nach Pakettyp. Beispielsweise enthält der Variable Header eines `PUBLISH`-Pakets den Topic-Namen. Der Header muss bestimmte Format- und Inhaltsanforderungen erfüllen, um gültig zu sein.
Wenn K gültige Kombinationen für den Variable Header existieren und die Länge des Headers L_p Bytes beträgt, dann ist die Wahrscheinlichkeit, dass der Variable Header gültig ist:

$$P(\text{gültiger Variable Header}) = \frac{K}{N^{L_v}}$$

Der Payload ist der flexibelste Teil des Pakets, mit den wenigsten Einschränkungen. Der Payload muss jedoch inhaltlich zu den Angaben im Fixed und Variable Header passen. Wenn die Länge des Payloads L_p beträgt, dann ist die Wahrscheinlichkeit, dass der Payload gültig ist:

$$P(\text{gültiger Payload}) = \frac{1}{N^{L_p}}$$

Die Wahrscheinlichkeit, dass ein vollständig zufällig generiertes MQTT-Paket gültig ist, ergibt sich aus dem Produkt der Wahrscheinlichkeiten der einzelnen Teile:

$$P(\text{gültiges Paket}) = P(\text{gültiger Fixed Header}) \times P(\text{gültiger Variable Header}) \times P(\text{gültiger Payload}) \quad (1)$$

Substituiert man die oben angegebenen Wahrscheinlichkeiten:

$$P(\text{gültiges Paket}) = \frac{M}{N^2} \times \frac{K}{N^{L_v}} \times \frac{1}{N^{L_p}}$$

Wenn der Fixed Header, der Variable Header und das Topic festgelegt sind und nicht gefuzzt werden, vereinfacht sich die Berechnung erheblich. In diesem Fall ist die Wahrscheinlichkeit, dass diese Teile korrekt sind, 1, da sie nicht zufällig generiert werden. Es bleibt nur die Wahrscheinlichkeit für den Payload:

$$P(\text{gültiges Paket bei festen Headern}) = \frac{1}{N^{L_p}}$$

Angenommen werden folgende Parameter:

- N = 256 (entsprechend 8-Bit-Byte-Werten)
- M = 16 (gültige Kombinationen für den Fixed Header)
- K = 256 (gültige Kombinationen für den Variable Header)
- Länge des Variable Headers: L_v = 2 Bytes
- Länge des Payloads: L_p = 10 Bytes

Dann ergibt sich die Wahrscheinlichkeit, dass ein gültiges Paket bei zufälligem Fuzzing generiert wird, zu:

$$P(\text{gültiges Paket bei zufälligem Fuzzing}) = \frac{16}{256^2} \times \frac{256}{256^2} \times \frac{1}{256^{10}} \approx 3{,}55 \times 10^{-27}$$

Im Gegensatz dazu ist die Wahrscheinlichkeit, dass ein gültiges Paket generiert wird, wenn Fixed Header, Variable Header und Topic festgelegt sind:

$$P(\text{gültiges Paket bei festen Headern}) = \frac{1}{256^{10}} \approx 1{,}00 \times 10^{-24}$$

Der Unterschied in den Wahrscheinlichkeiten ist beträchtlich. Beim zufälligen Fuzzing aller Teile liegt die Wahrscheinlichkeit, dass ein gültiges Paket generiert wird, bei ungefähr $3{,}55 \times 10^{-27}$. Wenn jedoch der Fixed Header, der Variable Header und das Topic festgelegt sind, steigt diese Wahrscheinlichkeit auf $1{,}00 \times 10^{-24}$. Dieser drastische Unterschied ist auf die Reduktion der Anzahl der zufällig zu generierenden Bytes zurückzuführen. Da der Fixed Header und der Variable Header spezifische Anforderungen haben und die Anzahl der gültigen Kombinationen begrenzt ist, erhöht das Festlegen dieser Felder die Wahrscheinlichkeit eines gültigen Pakets erheblich. Die Wahrscheinlichkeit hängt also stark davon ab, wie viele Teile des Pakets zufällig generiert werden und wie viele festgesetzt sind.

Berechnung der Wahrscheinlichkeit für ein valides Paket:

$$P(\text{gültiges Paket bei zufälligem Fuzzing}) = \frac{16}{256^2} \times \frac{256}{256^2} \times \frac{1}{256^{10}}$$
$$P(\text{gültiges Paket bei zufälligem Fuzzing}) = \frac{16}{65536} \times \frac{256}{65536} \times \frac{1}{1099511627776}$$
$$P(\text{gültiges Paket bei zufälligem Fuzzing}) = \frac{1}{4194304} \times \frac{1}{1099511627776}$$
$$P(\text{gültiges Paket bei zufälligem Fuzzing}) \approx 2.28 \times 10^{-25}$$

Um die Anzahl der Pakete zu berechnen, die erforderlich sind, um mit hoher Wahrscheinlichkeit (z.B. 99 %) mindestens ein gültiges Paket zu erhalten, verwenden wir die Formel:

$$n = \frac{\log(1 - \text{Wahrscheinlichkeit})}{\log(1 - P)}$$

Für eine Wahrscheinlichkeit von 99 %, dass mindestens ein Paket gültig ist:

$$n = \frac{\log(1 - 0.99)}{\log(1 - 2.28 \times 10^{-25})}$$

Da $\log(1 - x) \approx -x$ für sehr kleine Werte von x:

$$n \approx \frac{\log(0.01)}{-2.28 \times 10^{-25}} \approx \frac{-2}{-2.28 \times 10^{-25}} \approx 8.77 \times 10^{24}$$

Berechnung der Wahrscheinlichkeit für ein valides Paket:

$$P(\text{gültiges Paket bei festen Headern}) = \frac{1}{256^{10}}$$
$$P(\text{gültiges Paket bei festen Headern}) = \frac{1}{1099511627776}$$

Um die Anzahl der Pakete zu berechnen, die erforderlich sind, um mit hoher Wahrscheinlichkeit (z.B. 99 %) mindestens ein gültiges Paket zu erhalten:

$$n = \frac{\log(1 - 0.99)}{\log(1 - \frac{1}{1099511627776})}$$
$$\text{Da } \log(1 - \frac{1}{1099511627776}) \approx -\frac{1}{1099511627776}$$
$$n \approx \frac{\log(0.01)}{-\frac{1}{1099511627776}} \approx \frac{-2}{-9.05 \times 10^{-13}} \approx 2.21 \times 10^{12}$$

Diese Berechnungen zeigen, dass das Fixieren von Teilen des Pakets (wie Fixed Header, Topic und Variable Header) die Anzahl der benötigten Pakete zur Erzeugung eines gültigen Pakets erheblich reduziert. Dies ist besonders wichtig für die Effizienz von Fuzzing-Tests, da es die Anzahl der Tests verringert, die benötigt werden, um die gewünschten Ergebnisse zu erzielen.

Durch das Fixieren bestimmter Schlüsselparameter des MQTT-Pakets, wie dem Fixed Header und dem Topic, wird die Wahrscheinlichkeit, dass ein gültiges Paket generiert wird, signifikant

erhöht. Dies ist besonders wichtig für effizientes Fuzzing, da es die Anzahl der möglichen ungültigen Pakete reduziert und die Effektivität der Tests steigert.

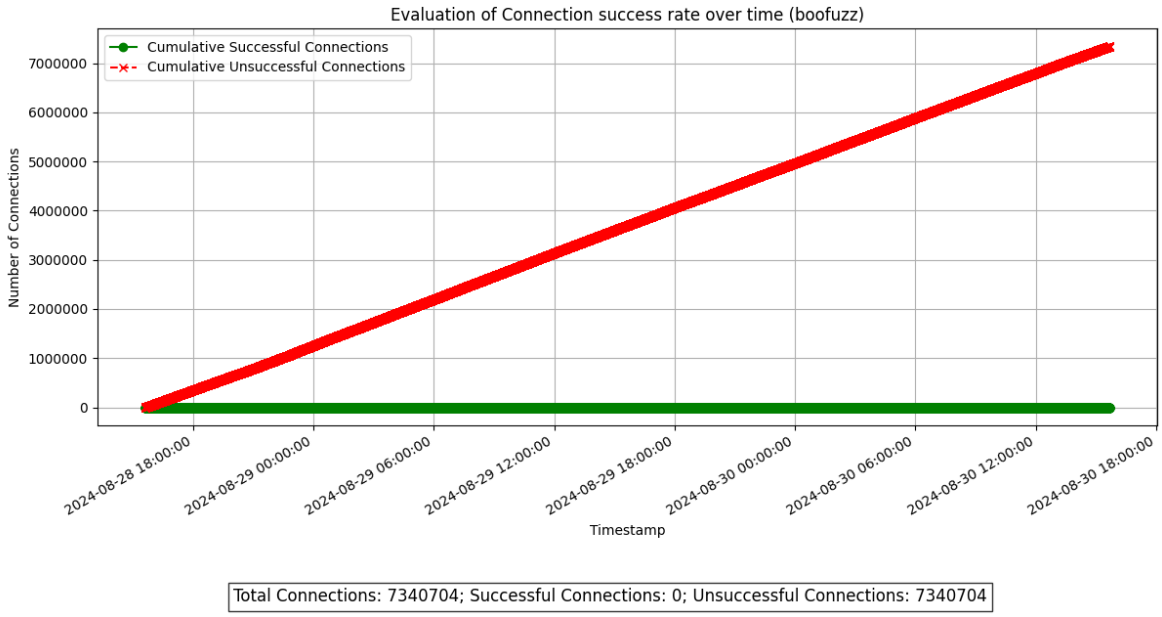

Abbildung 10: Vergleich der gültigen Pakete bei zufälligem Fuzzing mit boofuzz ohne statisch definierte Felder

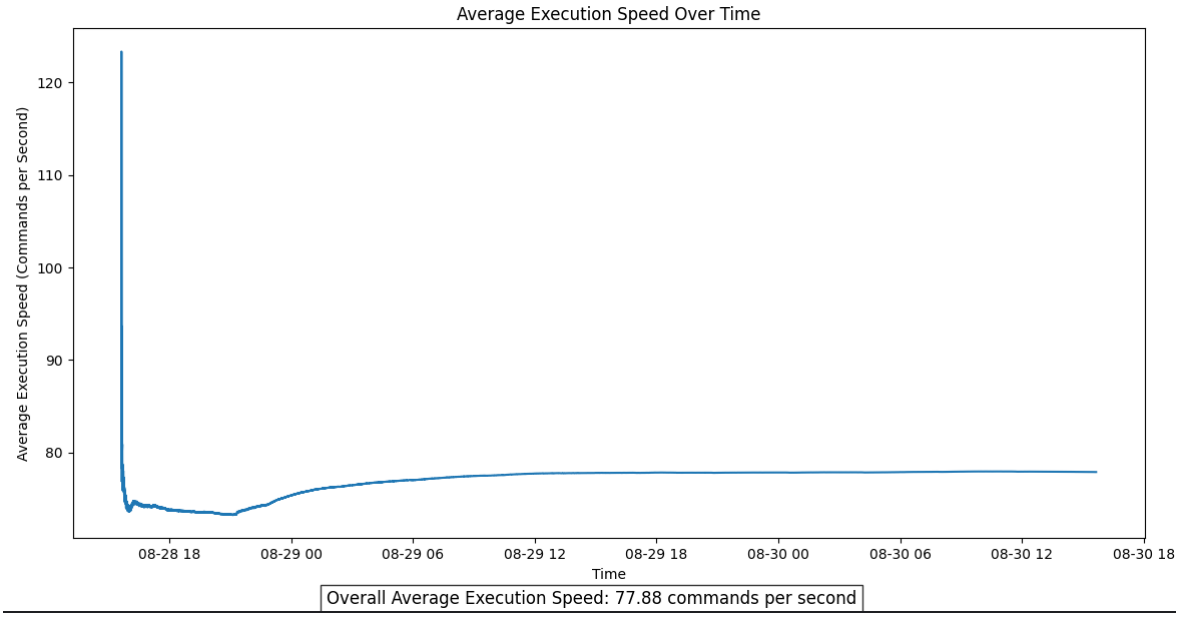

Abbildung 11: Durchschnittliche Ausführungsgeschwindigkeit von boofuzz ohne statisch definierte Felder

4.2.3 Analyse der gefundenen Bugs (Optional?)

4.3 Pulsar

Pulsar ist ein Fuzzer und wird zu der Familie der Blackbox-Fuzzer gezählt. Der Fuzzer wird dafür verwendet, um Internet Protokolle zu testen. Er ermöglicht es, ohne jedweden Quellcode anhand von gesammelten Traffic-Dumps eine State Machine eines ZuP zu erlernen. Das Besondere an diesem Fuzzer ist, dass er anhand von gesammelten Daten ein Model trainiert, welches es dem Untersucher des Programmes ermöglicht, das Programm zu simulieren und schlussendlich gezielt zu fuzzen. Bei der Simulation des Protokolls kann der Fuzzer sowohl die Rolle des Clients, als auch die Roller des Servers einnehmen. Gerade dieser Punkt ermöglicht es dem Untersucher des Programms große Flexibilität bei der Untersuchung zu erreichen. Somit kann ein großer Teil des Programms untersucht werden und möglichst tief in die Struktur des ZuP eingegriffen werden.
Das Training des Modells beruht auf der Analyse des eingefangenen Netzwerkverkehrs. Dabei werden verschiedene Nachrichten von sowohl Client- als auch Serverseite auf Byte-Ebene untersucht und ähnliche Strukturen extrahiert. Die daraus extrahierten Nachrichtensequenzen werden darauffolgend in einen endlich-deterministischen Vektor abgebildet. Die Zusammenkunft aus mehreren Vektoren ergibt ein Cluster. Anhand der entstandenen Cluster wird eine approximierte Abbildung der State-Machine des Protokolls geschlussfolgert. Bei der Erschließung der State Machine können die tatsächlichen Zustände nur teilweise erschlossen werden und ist somit von der Güte des gesammelten Netzwerkverkehrs abhängig. Der Netzwerkverkehr wird bei der Untersuchung von Pulsar annotiert, um die Nachrichten von Client und Server unterscheiden zu können. Zusammenhänge zwischen den Nachrichten werden anhand von bereits beobachteten Nachrichten erschlossen. Diese werden in Tupeln miteinander verglichen und miteinander assoziiert. Nachdem die Assoziation aller Mitteilungen abgeschlossen ist, besteht ein Markov Modell zweiter Ordnung. Dieses Modell wird in einem Graphen abgebildet und kann somit als eine Annäherung der State Machine interpretiert werden. Das Markov Modell wird im anschluss mit einem DFA (Deterministic Finite Automaton) minimiert. Der dabei entstandene DFA erlaubt es Analysten die Zustände manuell untersuchen zu können und gegebenenfalls die Zustände zu verfeinern und nachzuvollziehen.

4.3.1 Setup

Pulsar wurde mithilfe des von Pahl bereitgestellten Docker-Images installiert. Das Docker-Image wurde auf einem Ubuntu 20.04 LTS System installiert. Weitere Abhängigkeiten, welche nicht in der Installation der Docker-Images enthalten sind, wurden manuell installiert und

in einer `requirements.txt` Datei festgehalten. Für das Fuzzing mit Pulsar wird ein Netzwerkverkehr benötigt. Dieser Netzwerkverkehr wird in Form von pcap-Dateien gespeichert. Hierfür wurde ein Python-Skript entwickelt, welches MQTT spezifische Nachrichten an den Broker sendet und möglichst viele Zustände vom Broker erreicht. Die Nachrichten werden mithilfe des Tools `tcp` aufgezeichnet und in pcap-Dateien abgelegt. Die pcap-Dateien werden in einem Ordner `traffic` abgelegt. Für den Start werden zusätzlich die pcap-Dateien inm Ordner `traffic` über ein live-volume in den Docker-Container eingehängt. (TODO: Quelle) Alle gesammelten pcap-Dateien dienen für den weiteren Verlauf der Untersuchung. Pulsar verwendet die pcap-Dateien, um die State Machine zu erlernen.

4.3.2 Erkenntnisgewinn

4.4 Vergleich der erhobenen Metriken

5 Konklusion und Future Work

Anhang A Listings

Listing 1: Sanity-Checks des Brokers nach erhalten einer Publish Nachricht in der Funktion handle__publish aus srchandle_publish.c

```
  FIXME Intentionally commented out the sanity check for the payload
     ↪ length to be able to trigger buffer overflow
  if(msg->payloadlen){
     if(db.config->message_size_limit && msg->payloadlen > db.config->
        ↪ message_size_limit){
        log__printf(NULL, MOSQ_LOG_DEBUG, "Dropped␣too␣large␣PUBLISH␣
           ↪ from␣%s␣(d%d,␣q%d,␣r%d,␣m%d,␣'%s',␣...␣(%ld␣bytes))",
                   context->id, dup, msg->qos, msg->retain, msg->
                      ↪ source_mid, msg->topic, (long)msg->payloadlen);
        reason_code = MQTT_RC_PACKET_TOO_LARGE;
        goto process_bad_message;
     }
     msg->payload = mosquitto__malloc(msg->payloadlen+1);
     if(msg->payload == NULL){
        db__msg_store_free(msg);
        return MOSQ_ERR_NOMEM;
     }
     /* Ensure payload is always zero terminated, this is the reason
        ↪ for the extra byte above */
     ((uint8_t *)msg->payload)[msg->payloadlen] = 0;

     if(packet__read_bytes(&context->in_packet, msg->payload, msg->
        ↪ payloadlen)){
        db__msg_store_free(msg);
        return MOSQ_ERR_MALFORMED_PACKET;
     }
  }
```

Listing 2: Buffer Overflow in der Funktion handle__connect der Quellcodedatei src/handle_connect.c

```
int connect__on_authorized(struct mosquitto *context, void *
   ↪ auth_data_out, uint16_t auth_data_out_len){
  if(context->will) {
    log__printf(NULL, MOSQ_LOG_DEBUG, "Will␣message␣specified␣(%ld␣
       ↪ bytes)␣(r%d,␣q%d).",
         (long)context->will->msg.payloadlen,
         context->will->msg.retain,
         context->will->msg.qos);
  // FIXME: Intentional buffer overflow
         char vuln_buffer[512];
```

```
        memcpy(vuln_buffer, context->will, context->will->msg.
            ↪ payloadlen);

        log__printf(NULL, MOSQ_LOG_DEBUG, "\t%s", context->will->msg.
            ↪ topic);
  }
}
```

Listing 3: Buffer Overflow in der Funktion `handle__publish` in der Quelldatei `src/handle_publish.c` des Mosquitto Brokers

```
int handle__publish(struct mosquitto *context){
    [...]
    rc = mosquitto_acl_check(context, msg->topic, msg->payloadlen, msg->
        ↪ payload, msg->qos, msg->retain, MOSQ_ACL_WRITE);
        if(rc == MOSQ_ERR_ACL_DENIED){
            log__printf(NULL, MOSQ_LOG_DEBUG,
                "Denied␣PUBLISH␣from␣%s␣(d%d,␣q%d,␣r%d,␣m%d,␣'%s',␣...␣
                    ↪ (%ld␣bytes))",
                context->id, dup, msg->qos, msg->retain, msg->source_mid
                    ↪ , msg->topic,
                (long)msg->payloadlen);
            reason_code = MQTT_RC_NOT_AUTHORIZED;
            goto process_bad_message;
        }else if(rc != MOSQ_ERR_SUCCESS){
    // FIXME intentional buffer overflow
            memcpy(vuln_buffer, msg->payload, msg->payloadlen);
            db__msg_store_free(msg);
            return rc;
        }
    [...]
}
```

Listing 4: Memory Leak in der Funktion `int connect__on_authorised` der Quellcodedatei `src/handle_subscribe.c` des Mosquitto Brokers

```
    int handle__subscribe(struct mosquitto *context){
        [...]
        if(!slen){
                log__printf(NULL, MOSQ_LOG_INFO,
                    "Empty␣subscription␣string␣from␣%s,␣disconnecting.
                        ↪ ",
                    context->address);
                mosquitto__free(sub);
                mosquitto__free(payload);
                return MOSQ_ERR_MALFORMED_PACKET;
            }
            if(mosquitto_sub_topic_check(sub)){
```

```
            log__printf(NULL, MOSQ_LOG_INFO,
                  "Invalid␣subscription␣string␣from␣%s,␣
                     ↪ disconnecting.",
                  context->address);
            mosquitto__free(sub);
            mosquitto__free(payload);
// FIXME - this is a memory leak use after free
            strcpy(sub, "Use␣After␣free");
            return MOSQ_ERR_MALFORMED_PACKET;
         }
    [...]
}
```

Literatur

[1] OWASP. *Static Code Analysis*. URL: https://owasp.org/www-community/controls/Static_Code_Analysis (besucht am 30. 08. 2024).

[2] science direct. *Black-Box Testing*. URL: https://www.sciencedirect.com/topics/computer-science/black-box-testing (besucht am 30. 08. 2024).

[3] Andre Coulter. *Graybox Software Testing*. URL: https://www.cleanscape.net/stellar/docs_lib/paper_graybox.pdf (besucht am 30. 08. 2024).

[4] OWASP Foundation. *Fuzzing*. URL: https://owasp.org/www-community/Fuzzing (besucht am 11. 03. 2024).

[5] Maialen Eceiza, Jose Luis Flores und Mikel Iturbe. „Fuzzing the Internet of Things: A Review on the Techniques and Challenges for Efficient Vulnerability Discovery in Embedded Systems“. In: *IEEE Internet of Things Journal* 8.13 (2021), S. 10390–10411. DOI: 10.1109/JIOT.2021.3056179. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9344712 (besucht am 11. 03. 2024).

[6] Hongliang Liang. *Fuzzing: State of the Art*. URL: https://wcventure.github.io/FuzzingPaper/Paper/TRel18_Fuzzing.pdf (besucht am 30. 08. 2024).

[7] ADAM Greene Michael Sutton und Pedram Amini. *Fuzzing - Brute Force Vulnerability Discovery*. URL: https://github.com/mangonugen/CySecBooks/blob/master/Fuzzing%20Brute%20Force%20Vulnerability%20Discovery.pdf (besucht am 30. 08. 2024).

[8] Michal Zalewski. *Instrumentieren eines Programms mit AFL*. URL: https://github.com/google/AFL/blob/master/docs/technical_details.txt#L23-L76 (besucht am 11. 03. 2024).

[9] Zhao B. Li J. und Zhang C. *Fuzzing: a survey*. URL: https://doi.org/10.1186/s42400-018-0002-y (besucht am 30. 08. 2024).

[10] Yuqi Chen u. a. „Learning-Guided Network Fuzzing for Testing Cyber-Physical System Defences“. In: *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2019, S. 962–973. DOI: 10.1109/ASE.2019.00093. URL: https://arxiv.org/pdf/1909.05410 (besucht am 30. 08. 2024).

[11] Marcel Böhme u. a. „Directed Greybox Fuzzing“. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’17. Dallas, Texas, USA: Association for Computing Machinery, 2017, S. 2329–2344. ISBN: 9781450349468. DOI: 10.1145/3133956.3134020. URL: https://mboehme.github.io/paper/CCS17.pdf.

[12] Andrew L. Russell. *Rough Consensus and Running Code and the Internet-OSI Standards War*. URL: https://web.archive.org/web/20191117080112/https://www2.cs.duke.edu/courses/common/compsci092/papers/govern/consensus.pdf (besucht am 01. 09. 2024).

[13] *TCP Protocol Specification RFC 793*. URL: https://www.rfc-editor.org/rfc/rfc793#section-3.4 (besucht am 01. 09. 2024).

[14] kernel.org. *man tcp(7)*. URL: https://www.man7.org/linux/man-pages/man7/tcp.7.html (besucht am 01. 09. 2024).

[15] OASIS. *MQTT Version 3.1.1*. URL: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html (besucht am 01. 09. 2024).

[16] MQTT. *MQTT man page*. URL: https://mosquitto.org/man/mqtt-7.html (besucht am 01. 09. 2024).

Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe; die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht. Die Arbeit wurde nach meiner besten Kenntnis bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Hof, den 11.09.2024

Sebastian Peschke